

Bachelor's Thesis (Academic Year 2026)

High-Throughput Optical Testbed Simulation with Rust

Keio University
Faculty of Environment and Information Studies

Yutaro Yoshii

July 17, 2026

Abstract

For a long time, quantum computing has been exclusively the domain of physicists. However, in recent years, as the technology matures to enable a real quantum network to distribute entanglement across remote places, there has been a growing need to solve the associated engineering challenges. In order to prove the stability of such a complex system, one needs not only theoretical and physical design and validation, but also the systems engineering in handling such a large amount of data, such as one coming from physical sensor equipment as single photon detectors. In order to prove the robustness of such system under load under repeatable condition, integrated testing on a high throughput simulation platform is indispensable.

While there have been considerable effort in establishing and validating protocol level simulation, such as QuISP [2], SeQUeNCe [23], and NetSquid [9], there is a general lack of attention to high-volume, noisy, physically correct sensor data generation that tests the capability of operational software and hardware stack. There has been small number of physically correct, photonic level simulators, such as one by Zhang et al [3], but they do not produce nearly large enough amount of data required to stress test a performance critical software-hardware stack in production. This simulation software attempts to address this need by introducing a highly parallelizable batched discrete event simulation architecture, which achieves 13% of real-time speed on a memory-interface-memory entanglement swap link simulation by processing tens of millions of wave packet entities every second, achieving the throughput of 5 million time tag per second at the end of the pipeline. At the same time, by cleanly extracting the quantum state representation, the quantum states can be computed and collapsed with a suitable processing routine fit for the problem definition of each experiment and testing, providing enough primitives to simulate physically accurate, high fidelity quantum state evolution. As a proof of this concept, this work implements one of such a quantum processing routine using Monte Carlo method, which is capable of emulating the Hong-Ou-Mandel interference [24], however, this routine is interchangeable with a different simulation backend.

By bridging the gap between physical fidelity and engineering-scale performance, this simulation framework provides a scalable platform for developing and validating the software and hardware infrastructure of quantum networks.

Contents

1	Introduction	7
1.1	Motivation and Problem Statement	7
1.2	Scope	7
1.3	Contributions	8
1.4	Thesis Organization	8
2	Background	9
2.1	Quantum States	9
2.1.1	Discovery of Quantum States	9
2.1.2	Vector Space and Operations	9
2.1.3	Hilbert Space	10
2.1.4	Basis Vectors	11
2.1.5	State Vectors	11
2.1.6	Unitary Transformation	12
2.1.7	Dirac's Bra-Ket Notation	12
2.2	Quantum Computing	13
2.2.1	Entanglement and Bell States	13
2.2.2	Quantum Gates	13
2.2.3	Entanglement Distribution	15
2.2.4	Telegate and Teledata	15
2.2.5	Entanglement Swapping	15
2.3	Photonics	16
2.3.1	Photonic Qubits and Polarization Encoding	16
2.3.2	Testbed Setup	16
2.4	Testbed Simulation	17
2.4.1	Discrete Event vs. Continuous Time	17
2.4.2	Optimization Strategies	18
2.5	Comparison of Quantum State Representation	18
2.5.1	Jones Calculus	18
2.5.2	Truncated Fock Space	19
2.5.3	Wigner Functions	19
2.5.4	Scattering Matrix	20
3	Related Works	21
3.1	Quantum Network Simulators	21
3.2	Photonic and Qudit Simulation Frameworks	22

3.3	Theoretical Foundations	22
4	Photonic State Representation	23
4.1	Relevant Properties of the Photonic State	23
4.2	Degree of Freedom based basis representation	23
4.2.1	Basic Representation of a mode	24
4.2.2	Qubit vs Qudit representation	27
5	Error Representation	28
5.1	Types of Errors	28
5.1.1	Practical Error Modes in Photonics	28
5.2	Derivation of the Process Matrix χ	29
5.2.1	Density Matrix	29
5.2.2	CPTP Map and Kraus operators	30
5.3	Generalized Kraus representation	30
5.3.1	Vectorization of matrices	31
5.4	In-memory representation	31
5.4.1	Coherent and loss operators	31
5.4.2	Incoherent Error Operators	32
6	Representation Optimization	33
6.1	Occupied-Mode Reduction	33
6.2	The Representation as a Contract	34
7	Architectural Overview	35
7.1	Concurrency Architecture	35
7.1.1	Centralized State Storage and Locking Registry	35
7.1.2	Node Thread Lifecycle and Message Queues	39
7.2	Island Extraction and Delegated Collapse	39
7.3	Physics Architecture and Representations	41
7.3.1	Wavepacket and Island Mechanics	41
7.3.2	Component Models	46
7.3.3	Temporal disjointness	49
7.4	Configuration Ergonomics	49
8	Experiment Setup	52
8.1	Hardware and Runtime	52
8.2	Telemetry Context	53
8.3	Topology	53

9	Experiment and Evaluation	55
9.1	Arm Sweep Experiment	55
9.2	Arm Sweep Experiment – Result	55
9.3	Core Sweep Experiment	58
9.4	Memory Usage	58
10	Conclusion	61
10.1	Future Work	62

List of Figures

2.1	Four qubit quantum fourier transform. This circuit applies discrete fourier transform on the input state vector	15
2.2	Overview of the Moonshot Nagayama testbed. Photon streams are routed according to the network protocol and measured collectively at a Bell State Analyzer (BSA) node, which consists of fiber beam splitters (FBS) and fiber-polarizing beam splitters (FPBS). Reproduced from [1].	17
7.1	Overall multi-threaded architecture. Quantum state lives in the central store, which can be partially locked to enable concurrent processing	36
7.2	Thread and channel model of the simulator. Each node runs on its own thread, and wavepacket batches flow downstream through bounded channels.	39
7.3	Delegated collapse pipeline. The concurrency engine hands completed islands to the rayon thread pool, and the collapse results are written to Parquet files.	40
7.4	Circuit interpretation of an island. The operator DAG over the virtual mode indices reads as a qutrit circuit over $ H\rangle$, $ V\rangle$, and $ \text{vac}\rangle$	44
9.1	Detected packet rate as a function of the arm length difference, with ideal detectors ($\eta = 1$, no dark counts, 1 ns dead time).	57
9.2	Detected packet rate as a function of the arm length difference, with realistic detectors ($\eta = 0.9$, 100 s^{-1} dark counts, 20 ns dead time).	57
9.3	Sustained event rate as a function of core count, with realistic detectors ($\eta = 0.9$, 100 s^{-1} dark counts, 20 ns dead time). The dashed line marks the single-core QuISP baseline of 1.12×10^7 packets per second, measured in a zero-noise link-tomography scenario on the same machine.	59
9.4	Heap consumption with the parquet writer enabled.	60
9.5	Heap consumption with the parquet writer removed. This run demonstrates that the entire core routine of the simulation, excluding the write buffer, can fit inside L3 cache.	60

Listings

7.1	The WavePacket struct and its edge helpers	43
7.2	The IslandOfInteraction struct and its readiness check	44
7.3	Island merge and tombstone creation in the interaction store	45
7.4	Temporal indistinguishability of two wave packets	47
7.5	The SPD claimer thread loop	48
7.6	The SPD builder. Defaults model the laboratory detector; each setter over-rides one parameter.	50
8.1	Telemetry metric structs	53
8.2	HOM Topology Setup	54

List of Algorithms

1	Batch reservation on the interaction store. One mutex acquisition reserves every cell a batch needs; all state mutation then runs without the lock. Invariant: a cell is in state LOCKED if and only if exactly one slice holds it.	37
2	Island merge at a two-port component, with tombstone forwarding. Invariant: a tombstone's reference count equals the number of wave packets still in flight that point to the old handle; the cell is retired exactly when the count reaches zero.	38
3	Detector-side termination protocol. A detector never removes wave packets from an island: removal would break a later merge. It only counts terminations. Invariant: an island is collapse-ready if and only if its terminated count equals its active-packet count.	41
4	Monte-Carlo collapse of one island. Phase 1 translates the recorded operator list into a consumer map; phase 2 pushes photons through it. $\mathcal{I}(w_1, w_2)$ is the pairwise indistinguishability of (7.10).	42
5	Lazy pair generation at the EPPS. Instead of testing every pump pulse (Poisson process, one wake-up per pulse), the source samples the number of pulses until the next success from the geometric distribution. Both formulations produce the same statistics; the lazy one costs one sample per emitted pair.	48
6	Classical detector model, applied by the claimer thread to the claimed click stream. Detection times are sampled from each packet's Gaussian envelope, so the claimed stream is only near-sorted and is sorted first. lastClick persists across batches. Defaults: $\eta = 0.9$, $r_{\text{dark}} = 100 \text{ s}^{-1}$, $\tau_{\text{dead}} = 20 \text{ ns}$ (non-paralyzable).	51
7	Arm-sweep experiment	55
8	Core-sweep experiment	58

Chapter 1

Introduction

1.1 Motivation and Problem Statement

Simulating quantum networks accurately requires both high fidelity and efficient integration with downstream data-processing pipelines. Existing simulation frameworks often suffer from poor integration and low throughput, which limits their usefulness for long-duration experiments. In particular, real-time validation of error characterization, monitoring, and compensation routines in the Q-Fly testbed requires simulations that can process a large number of photon events efficiently while remaining compatible with the rest of the system. Therefore, a new simulation framework is needed that can achieve both high performance and seamless integration with the existing software and hardware stack.

1.2 Scope

In order to achieve a high throughput, certain tradeoffs were made in regards to the correctness of the simulation. Previous protocol-oriented quantum simulation in the likes of QuISP and SeQUeNCe rely heavily on a transaction-based order-critical logic. However, as the stated goal of this simulation is to model the statistical behavior of a large number of photons, a batch-based multi-threaded approach has been introduced to improve the system throughput. In return, the simulation sacrifices responsiveness in the nanosecond-regime, meaning that control signals, as enacted upon virtualized components such as switches and waveplates, have to conform to the batching granularity of around 20 microseconds of simulated time. However, this still proves sufficient for applications such as validation of compensation scheme, such as polarization compensation, as their update frequency is usually much lower than what is required by the protocol.

In addition to the constraints imposed by the batched execution, this work operate on further assumptions in regards to physical states, in order to better model the large scale group behavior of the photonic packets.

- Wave packets are gaussian-shaped
- Wave packet shapes do not change mid-travel; no dispersion.

- Each wave packet contains exactly zero or one photon. Meaning, photonic number states need to be represented with multiple overlapping wave packet entities
- Beam splitters and other two-port components act as coherent operators on the underlying state. Incoherence is modeled exclusively by single-input single-output components
- The network topology should be a directed-asynchronous-graph (DAG). This prevents the persistence of large entangled states, which is expensive to model.

Through this assumptions, the simulation is able to limit the degrees of freedom, while keeping the simulated states reasonably realistic.

1.3 Contributions

This thesis presents a Rust-based optical testbed simulator that addresses the limitations of prior work. The simulator implements batched loop algorithms to optimize computation, resulting in improved cache utilization and higher throughput. In what it sacrifices at In addition, the simulator separates the tracking of entanglement from the evaluation of quantum states: self-contained islands of interaction are extracted from the stream of wavepackets in a concurrent manner, and their collapse into measurement results is delegated to dedicated matrix engines in an embarrassingly parallel fashion. Furthermore, the interaction record and the operator representation developed in this thesis together form a compact, transferable, universal operator format: a language-independent contract between the testbed simulation and the matrix engines that evaluate the quantum states. Only the in-process engine side of this contract has been implemented in this work. However, the sufficiency of the representation is established so that an external implementation, written in another language or running on another node of a cluster, is as straightforward as possible. By combining performance optimization with this separation of concerns, the simulator facilitates accurate and efficient evaluation of quantum network error characterization and compensation.

1.4 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 provides background on quantum networks, photonic qubits, and the testbed that this work aims to simulate. Chapter 3 reviews existing simulation frameworks and the theoretical works that this thesis builds upon, highlighting their limitations and motivating the need for the proposed framework. Chapters 4 through 6 develop the representation of photonic quantum states and their errors, and the optimizations that make those representations computationally efficient. Chapter 7 describes the concurrency architecture of the high-throughput simulator, including the extraction of islands of interaction and the delegation of their collapse to parallel matrix engines. Chapters 8 and 9 describe the experiment setup and evaluate the simulator’s throughput. Finally, Chapter 10 summarizes the conclusions and outlines directions for future work.

Chapter 2

Background

2.1 Quantum States

2.1.1 Discovery of Quantum States

In 1902, Philipp Lenard has observed, that the speed of electrons, at the time known as the cathode ray, that is emitted from a material exposed to the ultraviolet radiation, is independent from the intensity of the radiation [29]. Later, Albert Einstein has described this phenomena by theorizing that the energy of light is distributed in a discontinuous manner across space, and that they consists of discrete packets [30], which is today known as photons. In some aspects, especially in terms of matter-light interaction, light exhibits particle-like characteristics. However, in other cases, light often exhibits wave-like properties. In classical mechanics, when two waves interact, they interfere and create a superposition. These superposition can be expressed as linear combination of the elementary waves, called basis. In quantum mechanics, particle states and photons are described as a superposition of these basis states, that are mathematically expressed as orthogonal element in the space of all possible configuration, called the Hilbert space. These state combination can be expressed using Dirac's ket notation.

2.1.2 Vector Space and Operations

In order to formalize the linear property of superposition, we need concepts from linear algebra to give a mathematical form to the underlying physical states. In linear algebra, a vector must satisfy some constraints in order to qualify as a vector space, which is called the vector axiom.

Let $u, v, w \in V$ be vectors in a vector space V , and scalars $a, b \in F$ in a field F . To qualify as a vector space, V must satisfy the following eight axioms:

Axiom	Condition
Associativity of vector addition	$u + (v + w) = (u + v) + w$
Commutativity of vector addition	$u + v = v + u$
Identity element of vector addition	There exists an element $0 \in V$, called the zero vector, such that $v + 0 = v$ for all $v \in V$.
Inverse elements of vector addition	For every $v \in V$, there exists an element $-v \in V$, called the additive inverse of v , such that $v + (-v) = 0$.
Compatibility of scalar multiplication with field multiplication	$a(bv) = (ab)v$
Identity element of scalar multiplication	$1v = v$, where 1 denotes the multiplicative identity in F .
Distributivity of scalar multiplication with respect to vector addition	$a(u + v) = au + av$
Distributivity of scalar multiplication with respect to field addition	$(a + b)v = av + bv$

Elements from a vector space can be transformed using a linear operator. In the context of quantum physics, this transformation can represent state evolution, and it is dimension preserving, so $T : V \rightarrow V$, however, underlying mathematic notation is more general, generally described as $T : V \rightarrow V$, and this must satisfy the following constraint:

$$T(au + bv) = aT(u) + bT(v) \quad (2.1)$$

The consequence of this is that the physical property of superposition is preserved during state evolution. If a system is a superposition of basis states, the result of the evolution must also be a superposition of linear operations applied individually to each basis states. Therefore, vector space proves sufficient to encode the wave-like nature of quantum mechanics.

2.1.3 Hilbert Space

Hilbert space $\mathcal{H}$ is one such vector space, consisting of complex scalar field where $F \equiv \mathbb{C}$

Since Hilbert space is a vector space, the linear combination of its element also falls within itself

$$af + bg \in \mathcal{H} \quad (2.2)$$

given $f, g \in \mathcal{H}$ and $a, b \in \mathbb{C}$.

Inner product within hilbert space (a, b) is also defined, with the following constraints:

Linilarity

$$(f, ag + bh) = a(f, g) + b(f, h) \quad (2.3)$$

where $f, g, h \in \mathcal{H}$, $a, b \in \mathbb{C}$

Hermitian Symmetry

$$(f, g) = (g, f)^* \quad (2.4)$$

Positive Semidefiniteness

$$(f, f) \geq 0 \quad (2.5)$$

$$(f, f) = 0 \Leftrightarrow f = \mathbf{0} \quad (2.6)$$

Additional requirements for a hilbert space are completeness and separableness. These qualities are called Hilbert Axioms [31], and they define a valid hilbert space, where physical states can be mapped to.

2.1.4 Basis Vectors

As stated above, a vector space consists of elements $v \in V$ that satisfy the aforementioned axioms. However, for a finite dimensional space, it is possible to form the entire space V using linear combinations of a minimal set of orthogonal normalized vectors called a *basis*.

For a finite-dimensional space, if a basis $B = \{b_1, b_2, \dots, b_n\}$ spans V , any arbitrary vector $v \in V$ can be uniquely expressed as:

$$v = \sum_{i=1}^n c_i b_i \quad (2.7)$$

where $c_i \in F$ are scalar coefficients from the underlying field F .

2.1.5 State Vectors

As alluded above, the states of all matters can be expressed as a linear combination of discrete basis states that are orthogonal to each other. In order to map physical states into these mathematical constructs, the concept of **state vector** must be introduced. A state vector consists of a set of physically possible states called **basis states**, which are orthogonal to each other. In light, this is can be horizontal and vertical polarization component, and in ionized atoms, the basis states can be the different orbitals that an electron can occupy. A state vector $v \in \mathcal{H}$ must be normalized, such that $(v, v) = 1$, in order to keep it physically viable.

$v \in \mathcal{H}$ can be expressed as a linear combination of its basis states, which forms the basis vectors of the underlying Hilbert space $\mathcal{H}$. As a consequence of the linearity of the Hilbert space introduced in (2.3), the following equality should always hold

$$v = \sum_{i=1}^n (c_i b_i, c_i b_i) = 1 \quad (2.8)$$

Consequently, each term $(c_i b_i, c_i b_i)$ represents the probability that the state exists as b_1 .

2.1.6 Unitary Transformation

As previously described, vector space, and consequently Hilbert space, allows linear operations to be applied to its constituents. However, in order to keep the states physically realistic, the equality (2.8) must be satisfied through the state transformation, to this end, the concept of **unitary transformation** is introduced.

A unitary transformation U can be represented as a unitary matrix that satisfies the following quality:

$$UU^\dagger = I \quad (2.9)$$

Where $U^\dagger$ represents the conjugate transpose of the matrix U , and I represents the identity matrix.

An important quality of a unitary transformation is that it is reversible. Given a state vector v_0 and a unitary transformation U , and the state after transformation v_1 , the state transformation can be expressed as the following

$$v_1 = v_0 U \quad (2.10)$$

As a consequence of reversibility, the following must be also true

$$v_0 = v_1 U^\dagger \quad (2.11)$$

2.1.7 Dirac's Bra-Ket Notation

In order to easily differentiate between scalars, linear operators, and the vectors that they operate on, Dirac's bra-ket notation is introduced. $|v\rangle$ denotes a vector v in a vector space V

Key notations are described below

- A **ket** describes a vector in the relevant vector space. It is denoted in the form $|v\rangle$, it plays a role of operand in describing the quantum state evolution. Item-wise, it represents a column-vector
- A **bra** describes a linear form $f : V \rightarrow \mathbb{C}$. It is written as $\langle f|$. Item-wise, it describes a row-vector.
- Inner product (f, v) is denoted in the bra-ket notation as $\langle f|v\rangle$, the product of which is a complex number.
- Linear operators are often denoted as capital letters, and they represent a square complex matrix.

Dirac's bra-ket notation is useful when denoting physical states, such as polarization of light. For example, the quantum state of a photon involving orthogonal polarization modes H and V can be described as follows

$$|\psi\rangle = \alpha|H\rangle + \beta|V\rangle \quad (2.12)$$

2.2 Quantum Computing

The physical quantum states, as described above, can be abstracted as dual-rail encoding to form quantized bits, called **qubits**. They form a fundamental unit of quantum information theory to enable the gate-based computation [8], and the implementation of quantum algorithms, such as the Shor's algorithm [32]. In place of physical states, such as the polarization modes H or V , a numeric notation 0 and 1 is used inside the bra-ket notation to denote the state of a system of qubits.

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad (2.13)$$

where $\alpha, \beta \in \mathbb{C}$ and $|\alpha|^2 + |\beta|^2 = 1$. Quantum information is manipulated using quantum gates, which are unitary operations, allowing for complex transformations that enable quantum algorithms and protocols.

2.2.1 Entanglement and Bell States

Entanglement is a uniquely quantum phenomenon in which the states of two or more qubits become correlated in such a way that the state of one qubit cannot be described independently of the other. The simplest form of entanglement is the Bell states. One such example state is the $|\Phi^+\rangle$ state.

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (2.14)$$

Qualitatively, this state means, that when one of the two qubits are measured, the state of the other qubit can be fully deterministic, based on the result of the measurement. This phenomena is described by Albert Einstein as the "spooky action at a distance"

There are four bell states in total. These four bell states form basis states of the closed two-qubit Hilbert space, alongside with the computational basis states $|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$.

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (2.15)$$

$$|\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) \quad (2.16)$$

$$|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle) \quad (2.17)$$

$$|\Psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) \quad (2.18)$$

Bell states are central to quantum networking applications, including quantum teleportation and entanglement swapping.

2.2.2 Quantum Gates

Quantum gates are fundamental primitive that allows universal computation to be implemented on a quantum computer. A quantum gate operates on one or more qubit, and it

represents a unitary that operates on the Hilbert space formed by the tensor product of the individual participant qubit states. Basic set of gates are described below:

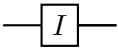
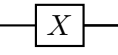
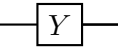
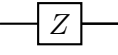
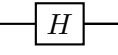
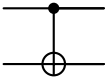
Gate	Circuit	Matrix
I Gate		$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$
Pauli- X Gate		$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$
Pauli- Y Gate		$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$
Pauli- Z Gate		$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$
H Gate		$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$
CNOT Gate		$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$
SWAP Gate		$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Table 2.1: Basic quantum gates represented as quantum circuit elements and their corresponding unitary matrices.

As a composite example, Figure 2.1 shows how these gates combine into the four-qubit quantum Fourier transform circuit.

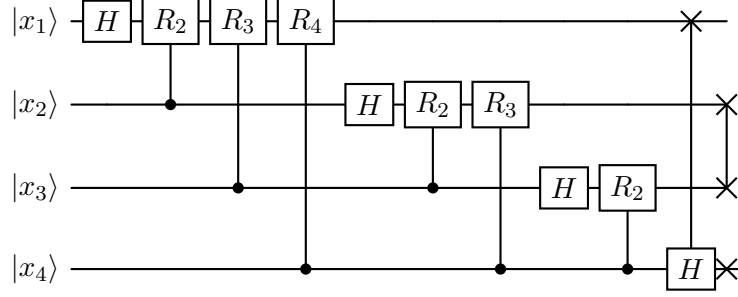


Figure 2.1: Four qubit quantum fourier transform. This circuit applies discrete fourier transform on the input state vector

2.2.3 Entanglement Distribution

In a quantum network, entanglement plays a pivotal role in enabling remote computational operations. As such, distributing entangled qubit pairs (ebits) constitutes the core challenge of designing a quantum network. The principal obstacle is photon loss, which grows with the length of the channel. As a consequence, the probability of directly transmitting a photon over a long link decays exponentially with distance, and direct transmission alone cannot sustain a usable entanglement rate. Quantum networks therefore introduce intermediate nodes, or repeaters, that perform entanglement swapping. A Bell state measurement (BSM) at a midpoint node projects two independently generated entangled pairs onto an entangled state spanning the full distance, allowing long-distance entanglement to be built from shorter elementary links.

2.2.4 Telegate and Teledata

In order to realize distributed quantum computing, there are two main approaches, Telegate and Teledata [33]. Telegate describes the architecture where entanglement resources are expended to emulate the execution of quantum gates, such as CNOT, across a distance. On the other hand, Teledata denotes the architecture where qubits are teleported across different quantum computers, in order to achieve the distribution of computation.

2.2.5 Entanglement Swapping

Entanglement swapping is the operation that both approaches ultimately rely on. Two entangled photon pair sources (EPPS) each emit a pair, and one photon from each pair is sent to a common midpoint node. There, the two photons interfere at a beamsplitter and are jointly measured in a Bell state measurement (BSM), projecting the two remaining photons onto an entangled state even though they never interacted. The success of this measurement depends critically on the indistinguishability of the two interfering photons, which is quantified by Hong-Ou-Mandel (HOM) interference [24]. For this reason, the testbed measures HOM interference as a direct diagnostic of photon indistinguishability and, by extension, of achievable swapping quality.

2.3 Photonics

In a photonic quantum network, entanglement between nodes is established using entangled photon pairs. Photons can be entangled in various degrees of freedom, including phase, number states, spatial modes, or polarization. In this work, we focus specifically on polarization-entangled photon pairs due to their widespread use in optical quantum networks and relative ease of manipulation and detection.

2.3.1 Photonic Qubits and Polarization Encoding

One intrinsic property of a photon that can be exploited to encode quantum information is its polarization. Polarization describes the orientation of the photon’s electric field and can be measured along any chosen axis. When a measurement is performed along a given axis, the photon’s polarization collapses to one of the basis states associated with that axis.

In the simplest case, the horizontal ($|H\rangle$) and vertical ($|V\rangle$) polarization states are chosen as the computational basis. An arbitrary photonic qubit can then be expressed as a superposition of these basis states:

$$|\psi\rangle = \alpha |H\rangle + \beta |V\rangle, \quad (2.19)$$

where $\alpha, \beta \in \mathbb{C}$ and $|\alpha|^2 + |\beta|^2 = 1$.

Alternative encoding schemes exist as well, such as time-bin encoding, path encoding, and frequency encoding, and may be selected depending on the specific requirements of the network or testbed infrastructure. Polarization encoding is particularly advantageous in fiber and free-space networks due to mature optical components for generation, manipulation, and measurement.

2.3.2 Testbed Setup

A quantum optical testbed provides a controlled environment for generating, manipulating, and detecting photonic qubits, enabling the experimental study of entanglement distribution and network protocols. In the Moonshot Nagayama project testbed, shown in Figure 2.2, entangled photon pairs are generated via spontaneous parametric down-conversion (SPDC) in PPLN crystals, pumped by 1 GHz pulsed lasers. Each pulse is spaced approximately 20 cm apart in the fiber, with a pair-generation probability of roughly 0.1% per pulse (≈ 1 pair/ μ s) [3].

After generation, the photons propagate through linear optical components, which introduce time-varying effects such as phase shifts, polarization rotations, loss, and decoherence. Finally, photons are detected using superconducting nanowire single-photon detectors (SNSPDs), producing high-resolution time-tagged events. Typical detector count rates range from 10^4 to 10^6 clicks per second per detector.

The main components of the testbed include:

- **Photon Sources:** SPDC sources and pulsed lasers provide single photons or entangled photon pairs for the network.

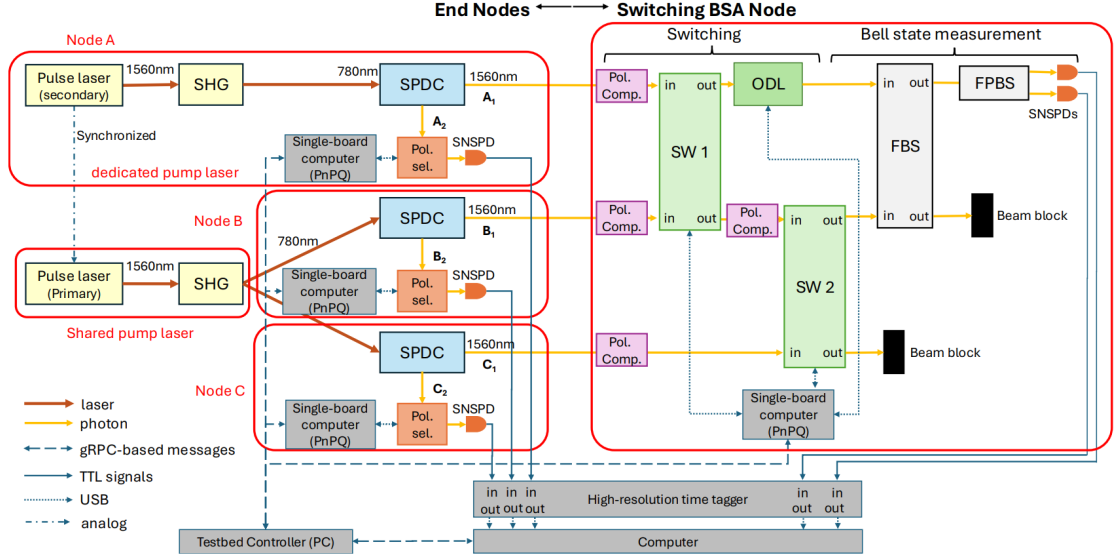


Figure 2.2: Overview of the Moonshot Nagayama testbed. Photon streams are routed according to the network protocol and measured collectively at a Bell State Analyzer (BSA) node, which consists of fiber beam splitters (FBS) and fiber-polarizing beam splitters (FPBS). Reproduced from [1].

- **Linear Optical Components:** Beam splitters, polarizers, and phase shifters allow deterministic manipulation of photonic qubits and implementation of network routing and measurement operations.
- **Detectors and Time-Tagging:** SNSPDs detect single photons and generate time-tagged data with sub-nanosecond resolution, enabling precise reconstruction of photon arrival times and network events.

2.4 Testbed Simulation

2.4.1 Discrete Event vs. Continuous Time

There are several different ways of conducting photonic simulation on an optical component. The first method is continuous time simulation. This method utilizes Maxwell's equations to model the electromagnetic behavior of light as it propagates through a device over continuous space and time. Continuous time simulations provide highly accurate physical representations of optical phenomena including interference, diffraction, reflection, and dispersion. However, these simulations are often computationally expensive and may require significant processing time and memory, especially for large-scale photonic systems.

An alternative approach is discrete event simulation. Rather than solving electromagnetic field equations continuously, discrete event simulation treats each wavepacket as a single entity with set parameters. Although this method does not simulate the full electromagnetic behavior of the wave and the medium within which it travels, this method provides an excellent means of simulation for media with known optical quality. As such,

this method provides a much more suitable simulation environment, on which this paper is based.

2.4.2 Optimization Strategies

The work proposes a novel architecture to simulate an optical test bed on a photonic level. Instead of event-queue-based discrete event simulation, the paper utilizes buffered photon wavepacket objects, and a separate data structure to keep track of the entanglement states between each wavepacket. This new architecture enables the longer-time-span simulation of the testbed behavior and helps validate quantum state estimation algorithms such as Distillation [27] and real-time fidelity tracking.

2.5 Comparison of Quantum State Representation

2.5.1 Jones Calculus

Derived from the Stokes vector, Jones Calculus plays a pivotal role in characterizing various linear optical components, such as linear polarizer, circular polarizer, and polarization dependent phase shifters, such as half wave plate, and quarter wave plate. Jones calculus provides a compact 2x2 matrix operator formalism for encoding these operations, which intersects with the photonic unitary operation, which serves as an elementary building block in the composition of a photonic qubit system.

At the most basic level, Jones calculus operates on the two component Jones vector, using 2x2 jones matrices. The following Jones vector represents an electromagnetic wave traveling in the Z direction, with amplitude and phase described as E and ϕ in direction x and y .

$$\begin{pmatrix} E_{0x}e^{i\phi_x} \\ E_{0y}e^{i\phi_y} \end{pmatrix} \quad (2.20)$$

A core observation is that this polarization state can be represented using the Dirac's ket notation

$$|\psi\rangle = E_{0x}e^{i\phi_x}|H\rangle + E_{0y}e^{i\phi_y}|V\rangle \quad (2.21)$$

With the corresponding operator definition given as a 2x2 matrix named Jones matrices, these representations form a foundation in bridging the gap between pure physical optical states to photonic quantum states. However, these representation lack in the ability to represent information in regards to non-polarization mode, such as wavelength or spatial mode, or operate on multimodal states, involving more than just 2 polarization states.

While Jones matrix representation is often the most popular means of representing the internal states of linear optical components. However, this representation alone leaves much to be desired, as it fails to capture the multi-modal nature of combined testbed quantum states. As a solution, many approaches have been tried in the past in order to

characterize and simulate the behavior of linear optical components, including truncated Fock state representations, phase-space distributions such as Wigner functions, and large-scale scattering matrix (S-matrix) formalisms.

2.5.2 Truncated Fock Space

Truncated Fock Space compresses the photonic number state Hilbert space into explicitly zero, one, or two photons, and defines matrix operators to determine the interaction between these three states. While this is useful in simplifying the representation of polarizing and unpolarizing beamsplitters, this sets forth the mandatory usage of a 9x9 density matrix, even in case of a single photon interaction with the beamsplitter. Furthermore, combination of certain input photonic states is by definition impossible to be represented, and therefore, the matrix acts as a non-unitary transformation. At a first glance, this representation promises the simplification of the beam splitter representation through a single matrix. However, this comes at a cost of leaky abstraction, with the necessity to handle the non-supported input states as special cases, as well as increased computational overhead, through the use of an unnecessarily large matrix for a single photon input state.

2.5.3 Wigner Functions

An alternative to the discrete number-state picture is the phase-space formulation, of which the Wigner function is the most widely used representative. Rather than expanding the state in a photon-number basis, the Wigner function $W(x, p)$ assigns a real-valued quasi-probability density to each point of the optical phase space spanned by the field quadratures x and p . Through the Weyl transform it stands in one-to-one correspondence with the density matrix, and therefore constitutes a complete description of the quantum state. For a single mode the distribution is supported on a two-dimensional plane; for an M -mode network it occupies a $2M$ -dimensional phase space.

The principal strength of this representation lies in the continuous-variable regime. Gaussian states, coherent, squeezed, and thermal states, possess Gaussian Wigner functions that are fully specified by a mean vector and a covariance matrix, and the action of passive linear optics on such states reduces to affine symplectic transformations of these moments. For Gaussian inputs under Gaussian operations the formalism is thus extremely compact and efficient.

This efficiency, however, does not extend to the states that are of primary interest to a photonic network. The single-photon and multi-photon number states that the present work seeks to model are non-Gaussian, and by Hudson’s theorem [25] any pure non-Gaussian state necessarily exhibits regions of negative quasi-probability. This negativity is the source of three difficulties. First, a negative $W(x, p)$ cannot be interpreted or sampled as an ordinary probability distribution, so the recovery of discrete photon-counting statistics, the very quantity a detector measures, becomes indirect and numerically delicate. Second, the natural objects of the testbed (definite mode occupation, discrete detection events, polarization-mode mixing) are awkward to express in a quadrature basis, producing a representation mismatch with the Jones-matrix calibration target. Third, and

most decisively, a faithful numerical treatment of a non-Gaussian state requires discretising phase space on a fine grid; for an M -mode system this grid is $2M$ -dimensional, and its memory cost grows exponentially with the number of modes. The Wigner function is therefore well suited to continuous-variable Gaussian optics, but ill suited to the discrete, multi-modal, number-resolved setting considered here.

2.5.4 Scattering Matrix

The scattering-matrix (S-matrix) formalism describes a linear optical subsystem at the level of its modes rather than its states. This in effect represents the Heisenberg picture where an operator is the main focus, rather than the states. Given an input operator $\hat{a}_{\text{in}}$, the output operator $\hat{a}_{\text{out}}$ can be calculated as follows:

$$\hat{a}_{\text{out},i} = \sum_{j=1}^M S_{ij} \hat{a}_{\text{in},j}, \quad (2.22)$$

An important observation here is that this relation also holds for creation operators, representing the photon counts at the input ports and output ports. This in effect allows the scattering matrix to represent beam splitters, wave plates, and interferometers such as MZI. Viewed in this light, the Jones matrix is nothing other than the 2×2 scattering matrix acting on the two polarization modes of a single spatial channel, which makes the S-matrix a natural candidate for a unified, component-level description.

Several properties make this formalism attractive. The matrix is compact, its dimension being fixed by the number of modes and independent of the photon number; it is composable, so that an arbitrary network reduces to a single product matrix; and it is unitary for lossless passive elements, guaranteeing conservation of probability. These are precisely the properties that the Jones representation enjoys for an isolated component, now extended to the multi-mode setting.

One limitation of S-matrix is that it cannot be directly used in a multi-body photonic states; Permutations of $n \times n$ submatrices need to be computed for an n -photon input state, which is a $\#P$ -hard problem [26]. A Hong-Ou-Mandel coincidence, for example, must require the evaluation of 4×4 matrix, even before taking into account of polarization. However, since the probability of photonic events involving more than two photons are extremely limited, this compounding effect can be somewhat circumvented as a result of amortization. A second limitation is that a single unitary matrix cannot represent neither loss nor mode-dependent error. While on a single-mode settings, these errors are represented with the Kraus matrices, a multi-mode generalization of this would require a significant increase in complexity, as illustrated with the effect of compounding permutation. It is nevertheless this formalism that comes closest to the requirements of the present work. Its mode-operator basis, its compositional structure, and its direct continuity with the Jones matrix makes it suitable for the dynamic nature of the simulation architecture.

Chapter 3

Related Works

This chapter reviews the existing simulation frameworks that target quantum networks, as well as the theoretical works that this thesis builds upon, and positions the present work among them.

3.1 Quantum Network Simulators

QuISP [2] is an event-driven simulator of large-scale quantum networks built on top of OMNeT++. It targets the protocol behavior of networks spanning thousands of qubits over dozens of nodes, and to this end, it abstracts the physical layer into error models attached to qubits and links. This abstraction is what enables its scale. However, it also means that the photonic details of a testbed, such as the temporal shape of a wavepacket or the interference of two pulses at a beamsplitter, are outside of its scope. Quisp simulates each pump laser pulse as discrete event, and therefore unfit for large scale physically realistic simulation.

NetSquid [9] is a discrete-event simulator of quantum networks with modular models of the physical hardware. Notably, NetSquid tracks quantum states independently and merges them through tensor products only when an entangling operation occurs. This dynamic state partitioning is close in spirit to the island of interaction introduced in this work, and it establishes that tracking entanglement lazily, instead of maintaining one global state, is a recognized practice for scaling up quantum network simulation. The difference lies in the execution model: NetSquid processes events sequentially through a centralized event queue, while the present work streams batches of wavepackets through concurrently running component threads, and delegates the evaluation of the collected islands to parallel matrix engines. In addition, NetSquid’s shared states both merge and split dynamically over their lifetime, while the islands of this work only ever merge. This monotonicity is what keeps the concurrent locking protocol of the store simple.

Closest to this work is the module-based discrete-event simulator of quantum interconnects by Zhang et al. [10], which models the same class of testbed hardware as this thesis (fiber components, interference at a Bell state analyzer, and single photon detectors), and which has been experimentally validated against the Q-Fly prototype [1] with module-level deviations of 1–5% [3]. The present work is complementary to it: rather than focusing on

the faithfulness of individual modules, it focuses on the throughput and the concurrency required to generate realistic time tag streams over long simulated durations.

3.2 Photonic and Qudit Simulation Frameworks

On the circuit-model side, MQT Qudits [4] provides a language, compiler, and simulator for mixed-dimensional qudit systems, reflecting the growing sophistication of the encoding schemes available on physical platforms. The review of linear optical quantum computing by Kok et al. [11] summarizes the dual-rail photonic encoding and the treatment of photon loss that underlie photonic computation. These works target the gate and circuit abstraction rather than the continuous stream of pulses of a testbed, but they inform the state and error representations adopted in this thesis.

3.3 Theoretical Foundations

The representation theory that this thesis relies on is standard: the postulates of quantum mechanics on composite systems and the operator-sum representation of quantum operations are covered by Nielsen and Chuang [8], and a compact pedagogical treatment of CPTP maps, Kraus operators, and the Markovian master equation is given by Manzano [7]. The starting point of the error representation developed in Chapter 5 is the complete characterization of completely-positive trace-preserving maps on 2×2 density matrices (qubit channels) by Ruskai, Szarek, and Werner [6]. This thesis extends their analysis to a three-level photonic system with an absorbing vacuum level, which captures photon loss natively while retaining the qubit channel structure in the polarization block.

Chapter 4

Photonic State Representation

The primary purpose of a quantum network is to distribute entanglement between two nodes, therefore, at the highest level, a single Qubit density matrix representation as well as the corresponding error channels such as Pauli channel is often sufficient. However in order to simulate the exact behavior of the testbed as well as the specific error mode, such as depolarization and polarization dependent photon loss, as well as multi-photon entanglement mode generated by components such as a beam splitter or an imperfect MZI-based photonic switch, capturing the details specific to optical medium is essential to the accuracy of the simulation.

4.1 Relevant Properties of the Photonic State

At the most elementary level, our testbed experiments are based on polarization based encoding. However, in specific situation, such as modeling the output of the Hong-Ou-Mandel interference, just encoding the polarization basis as well as the time tag of the wave packets prove not to be sufficient, as the output modes in this case contain two photons, when the interference succeed. Furthermore, as the level of sophistication in the photonic encoding scheme increases [4, 28], considering non-polarization modes, such as temporal, spatial, photon number, and frequency mode are becoming increasingly relevant. Therefore, there is a need to represent all the relevant optical modes, while warranting the encoding and computational efficiency.

4.2 Degree of Freedom based basis representation

Fundamentally, quantum systems are realized as a superposition of physically possible states. This, in the photonic representation, includes the polarization mode $|H\rangle$ and $|V\rangle$, the photonic number states $|0\rangle$, $|1\rangle$, and $|2\rangle$, temporal mode $|t\rangle$ (often paired as $|early\rangle$ and $|late\rangle$ to form the time-bin qubits), chromatic mode, and spatial mode (which-path information). These collectively form the orthogonal basis for the photonic quantum state space $\mathcal{H}$, from which two states are usually selected to represent the dual rail encoding, or a qubit. From these degree of freedom, Jones calculus only accounts for the polarization mode $|H\rangle$ and $|V\rangle$. Therefore, we need a more general operator and representation to

enable the simulation of the states general photonic bases. Here, an important realization can be made, that temporal, chromatic, and spatial modes can be extracted as wave packets, which can be handled classically as routable entities, to be collapsed later using a dedicated quantum state simulator routine.

Since all of the constituent basis for a wave packet are orthogonal, all wave packets can be viewed as orthogonal basis, with single port, or multi-port operators operating on a selected subset of them.

4.2.1 Basic Representation of a mode

$$|wavepacket_{polarization}\rangle \quad (4.1)$$

Here, wavepacket contains information such as wavelength, number of photons, and time distribution. Which allows for the chaining of these modes to express a basis that constitutes a multi-wave-packet quantum state.

As an elementary example, take the output of the type 2 SPDC. The output canonically follows the format of $\frac{1}{\sqrt{2}}(|HV\rangle + |VH\rangle)$. This formalism implicitly encodes the spacial modes of the qubit as position of each basis (positional encoding). More explicitly, given the output spacial modes of A and B, this could be written as follows:

$$\frac{1}{\sqrt{2}}|A_H\rangle \otimes |B_V\rangle + \frac{1}{\sqrt{2}}|A_V\rangle \otimes |B_H\rangle \quad (4.2)$$

Here, Spatial mode A and B implicitly encode the wavelength, photon number states and time spreads. However, an important constraint is that these variables have limited spacial, temporal, as well as frequency uncertainties. Therefore, these three characteristics can be encoded into a single struct, and efficiently passed around in the simulation.

An important architectural decision in the making of this simulation was to separate the representation of spacial representation of the photons (**WavePackets**) and the entanglement between the associated wave packets as separate quantum state graph, bearing the name **IslandOfInteraction**

```

1 // The packet itself represents the spatial and temporal basis.
2 // Internal wavelength encodes an extra degree of freedom
3 pub struct WavePacket {
4     // Temporal basis
5     // Both in picoseconds, three sigma serves the
6     // cutoff for packet interference
7     pub time: Time,
8     pub time_sigma: u32,
9     // Chromatic basis
10    pub wavelength: f32,
11    pub wavelength_sigma: f32,
12    pub state_handle: u32,
13    pub snowflake: u32,
14 }

```

```

1 pub struct IslandOfInteraction {

```

```

2 // Linear optical components, such as EPPS, waveplates,
3 // beam splitters, and detectors are stored as operators
4 pub operators: SmallVec<[Operator; 13]>,
5 // in-flight packets are stored, for house keeping purposes
6 pub active_packets: ActivePacketStore,
7 // wave packets are represented as 16 bit virtual index,
8 // mapped down from 32 bit snowflake
9 mode_max: u16,
10 }

```

The operation on the qudits and the photonic quantum states can be roughly split into two categories, single port transfer, and multi-port transfer. The former of which will be realized through the application of Kraus Operators representing possibly erroneous permutation of the intended Jones Matrix in the CPTP operator space. As for the latter, the representation of multi-port components are supported through creation operator scatter matrix, which maps four input modes to an equal number of output modes, preserving the linearity of the operation.

Given a generalized beam splitter (both polarizing beam splitter, and unpolarizing beam splitter of arbitrary reflection ratio) with two inputs and two output could be represented using the following formula, given source channel A , B , and sink channel C , and D .

$$A_H^\dagger = M_{11}C_H^\dagger + M_{12}C_V^\dagger + M_{13}D_H^\dagger + M_{14}D_V^\dagger, \quad (4.3)$$

$$A_V^\dagger = M_{21}C_H^\dagger + M_{22}C_V^\dagger + M_{23}D_H^\dagger + M_{24}D_V^\dagger, \quad (4.4)$$

$$B_H^\dagger = M_{31}C_H^\dagger + M_{32}C_V^\dagger + M_{33}D_H^\dagger + M_{34}D_V^\dagger, \quad (4.5)$$

$$B_V^\dagger = M_{41}C_H^\dagger + M_{42}C_V^\dagger + M_{43}D_H^\dagger + M_{44}D_V^\dagger. \quad (4.6)$$

A 50/50 unpolarizing beamsplitter (BS) is therefore

$$A_H^\dagger = \frac{1}{\sqrt{2}}C_H^\dagger + \frac{1}{\sqrt{2}}D_H^\dagger, \quad (4.7)$$

$$A_V^\dagger = \frac{1}{\sqrt{2}}C_V^\dagger + \frac{1}{\sqrt{2}}D_V^\dagger, \quad (4.8)$$

$$B_H^\dagger = \frac{1}{\sqrt{2}}C_H^\dagger - \frac{1}{\sqrt{2}}D_H^\dagger, \quad (4.9)$$

$$B_V^\dagger = \frac{1}{\sqrt{2}}C_V^\dagger - \frac{1}{\sqrt{2}}D_V^\dagger \quad (4.10)$$

A 50/50 polarizing beam splitter (PBS) can also be expressed as the following creation operator formalism:

$$A_H^\dagger = C_H^\dagger, \quad (4.11)$$

$$A_V^\dagger = D_V^\dagger, \quad (4.12)$$

$$B_H^\dagger = D_H^\dagger, \quad (4.13)$$

$$B_V^\dagger = C_V^\dagger \quad (4.14)$$

Realize, that both of these can be represented as weights within 4x4 matrices

Beam Splitter (BS)

$$\begin{pmatrix} C_H^\dagger \\ C_V^\dagger \\ D_H^\dagger \\ D_V^\dagger \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix} \begin{pmatrix} A_H^\dagger \\ A_V^\dagger \\ B_H^\dagger \\ B_V^\dagger \end{pmatrix}. \quad (4.15)$$

Polarizing Beam Splitter (PBS)

$$\begin{pmatrix} C_H^\dagger \\ C_V^\dagger \\ D_H^\dagger \\ D_V^\dagger \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} A_H^\dagger \\ A_V^\dagger \\ B_H^\dagger \\ B_V^\dagger \end{pmatrix}. \quad (4.16)$$

Furthermore, this concept can be taken further to represent an MZI switch in its arbitrary state of transition.

A Cross Configuration

$$\begin{pmatrix} C_H^\dagger \\ C_V^\dagger \\ D_H^\dagger \\ D_V^\dagger \end{pmatrix} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} A_H^\dagger \\ A_V^\dagger \\ B_H^\dagger \\ B_V^\dagger \end{pmatrix}. \quad (4.17)$$

And lastly, a bar configuration of such switching component can trivially be expressed as an identity matrix.

A Bar Configuration

$$\begin{pmatrix} C_H^\dagger \\ C_V^\dagger \\ D_H^\dagger \\ D_V^\dagger \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} A_H^\dagger \\ A_V^\dagger \\ B_H^\dagger \\ B_V^\dagger \end{pmatrix}. \quad (4.18)$$

$$\rho_1 = T \rho_0 T^\dagger \quad (4.19)$$

$$\rho_0 \in \mathcal{H}_{\text{left}} \subset \mathcal{H}_{\text{global}} \quad (4.20)$$

$$\rho_1 \in \mathcal{H}_{\text{right}} \subset \mathcal{H}_{\text{global}} \quad (4.21)$$

This shows that effectively any two by two optical component can be expressed using the scattering matrix formalism. This can be generalized further into an $n \times n$ linear optical component, in the format

$$Modes_{\text{Result}} = S \times Modes_{\text{Initial}} \quad (4.22)$$

This can be reduced back to

$$\hat{a}_{\text{out},i} = \sum_{j=1}^M S_{ij} \hat{a}_{\text{in},j}, \quad (4.23)$$

4.2.2 Qubit vs Qudit representation

In the previous subsection, we have seen that linear optical components on a testbed, together with the photonic states that they operate on across time, can be represented as individual operators $\mathcal{O}$ operating on a selected bases from the testbed-global photonic Hilbert space $\mathcal{H}_{global}$. This however enforces no constraint to the fact that the Hilbert space and operators have to be power of 2 in dimension, nor that the operators neatly sit within imaginary boundaries drawn within the Hilbert space, as one might usually expect from a constituent of a qubit-based quantum computer. This comes down to the fact that the dual-rail encoding in qubit is an artificially enforced construct. While the introduction of qubit is useful as an abstraction in terms of algorithm design, complexity analysis, and error correction, they bare no usefulness in terms of purely simulating the behavior of a physical system. Removing these layer of abstraction reveals the fundamental building block of quantum operation, operator $\mathcal{O}$ and operand $\mathcal{H}$. It is proven, that combination of beamsplitters and phase shifters could represent an arbitrary $N \times N$ unitary [22]. This proves that it is possible to compose relevant subspace operators $\mathcal{O}|_{\mathcal{H}_{sub}}$ to realize an operator $\mathcal{O}_{global}$ that can represent arbitrary linear optical operation on a photonic quantum system. Since the selected elementary operators operate on 3 basis photonic polarization+vacuum basis, as well as 4 modal-polarization basis, they can be better abstracted as a traced operation on a bigger hilbert space expressed as a density matrix, rather than operating on discrete photonic qubits.

Chapter 5

Error Representation

In the previous section, we have described the representation of photonic quantum state through the degree of freedom representation, through which, any types of coherent error could be represented. In order to express the error in this space, we will be relying on the density matrix representation of the open Hilbert space $\mathcal{H}_{open}$, and by introducing several concepts from general representations of the CPTP quantum channels for single port linear optical components. While the scatter matrix and transfer matrix introduced in the previous chapter are able to explain coherent error mode, in order to map a pure state into a mixed state, we need a slightly more involved representation. In the classical operation regime, these mixed states can be operated by the means of Mueller Calculus, we need some adaptation to encode map between mixed quantum states [19]. Through the introduction of the χ matrix [20], it is shown that we can map this CPTP operation to 7 kraus operators

5.1 Types of Errors

In photonic qubits, there are multiple types of errors that we need to account for. The error modes can be largely split into three large categories: coherent errors, incoherent errors, and loss error. Coherent error and incoherent error are classified as being part of operational error, and they represent the inconsistency in the resulting photonic state after a transit through an optical system, when compared to the intended photonic state. Loss error meanwhile, as the name suggests, represent the absorption or leakage of light while in-transit. Together, they represent in concrete terms what can go wrong on a flying qubit.

5.1.1 Practical Error Modes in Photonics

Photonic systems, particularly those employed in quantum communication and high-precision sensing, are susceptible to various noise sources and hardware imperfections. These errors can be broadly classified into coherent, incoherent, and loss-based mechanisms.

(1) Coherent Errors

Coherent errors are characterized by systematic shifts that maintain the phase relationship of the optical signal but deviate from the intended state. Unlike stochastic noise, they do not average out, and their amplitudes can accumulate linearly across components [14].

- **Phase Drift:** Gradual changes in the optical path length or refractive index, typically driven by thermal fluctuations or mechanical vibrations, lead to phase instability.
- **Polarization Drift:** Environmental stressors, such as temperature gradients and physical stress on the fiber, cause the state of polarization (SOP) to rotate over time, necessitating active compensation.

(2) Incoherent Errors

Incoherent errors result from stochastic processes that lead to dephasing or the temporal broadening of wave packets, effectively reducing the visibility of interference.

- **Wave packet Dispersion:** This includes phase dispersion and fluctuations in arrival time or path length. Such dispersion leads to the temporal spreading of the pulse, reducing the temporal overlap required for high-fidelity interference.
- **Polarization Mode Dispersion (PMD):** Caused by asymmetries in the fiber core, PMD results in different polarization components traveling at different velocities, leading to pulse broadening and signal degradation [16].

(3) Loss Errors

Loss errors represent the irreversible departure of photons from the system, which directly impacts the signal-to-noise ratio and the efficiency of photonic protocols.

- **Fiber Loss:** Attenuation within the transmission medium due to Rayleigh scattering and absorption.
- **Coupling Loss:** Efficiency drops occurring at interfaces, such as fiber-to-chip coupling or component-to-component splicing, often due to mode-field diameter mismatches or misalignment.

5.2 Derivation of the Process Matrix χ

The evolution of a closed quantum system can be described as a unitary map. However, in a real quantum system, it is not possible to guarantee complete isolation with the rest of the world. Therefore, one needs a more elaborate representation of the state as well as the map between the states.

5.2.1 Density Matrix

In order to represent a quantum state that is not perfectly isolated, the density matrix formalism is used. A density matrix ρ expresses a statistical mixture of pure states $|\psi_i\rangle$

with classical probabilities p_i :

$$\rho = \sum_i p_i |\psi_i\rangle \langle \psi_i| \quad (5.1)$$

This makes it possible to describe both the coherent part of the state and the classical uncertainty introduced by the environment within a single object, which is exactly the situation of a photonic state traveling through a lossy, drifting testbed.

5.2.2 CPTP Map and Kraus operators

A set of Kraus operators $\{K_i\}$ represents the map between an input density matrix and an output density matrix. Any physical evolution of an open quantum system, including the error channels of interest in this work, can be written as a completely positive trace preserving (CPTP) map in this operator-sum form, which follows from the Choi-Kraus theorem [7]. The same reference also justifies the discretization used throughout this simulator: under the Markovian (memoryless environment) assumption, a continuous noisy evolution can be sliced into a sequence of CPTP maps, one per component traversal.

5.3 Generalized Kraus representation

A Quantum channel in a Hilbert space $\mathcal{H}_n$ of dimension n can be expressed with the following expression:

$$\rho_1 = \sum_i^{n^2} K_i \rho_0 K_i^\dagger \quad (5.2)$$

Here, the following constraints must be satisfied [7]

$$\sum_i^{n^2} K_i^\dagger K_i = I_n \quad (5.3)$$

where ρ_0 and ρ_1 are density operators on $\mathcal{H}_n$, and at most n^2 Kraus operators are needed.

Fundamentally, a set of Kraus operators $\{K_i\}$ describes the most general representation of the evolution of a mixed quantum state. A combination of quantum channels $\{C_i\}$ forms a subspace inside all possible CPTP maps allowed under the given dimension by $\{K_i\}$. For a channel on a d -level system, at most d^2 Kraus operators are needed, as the minimal number of Kraus operators equals the rank of the Choi matrix of the channel [6].

The structure of such channels on a single qubit (2×2 density matrices) has been fully characterized by Ruskai et al. [6], who reduce every qubit channel to a canonical form of rotations around a signed-scaling map and derive the explicit conditions for complete positivity. The present work extends this analysis to the three-level system $\{|H\rangle, |V\rangle, |vac\rangle\}$ with an absorbing vacuum level. Since no passive optical process can bring a photon back out of the vacuum, and the optical superselection rule forbids coherent superpositions between different photon-number sectors in the absence of a shared phase reference [12],

the Kraus operators of a linear section are constrained to a lower block-triangular form. These operators span a 7-dimensional operator subspace, with the basis given in the following sections, so the process matrix χ of the channel reduces from 9×9 to 7×7 , and at most 7 Kraus operators are needed instead of the 9 expected for a general qutrit channel. The upper 2×2 block of each Kraus operator embeds a trace-non-increasing qubit map of exactly the kind characterized in [6]. When the loss is polarization independent, renormalizing by the survival probability recovers a trace-preserving qubit channel to which their analysis applies directly. The loss transitions themselves correspond to the truncation of the bosonic pure-loss (amplitude damping) channel to the 0- and 1-photon subspace [13], and the same no-return, block-structured treatment appears in the modeling of leakage errors on other hardware platforms [15].

5.3.1 Vectorization of matrices

An arbitrary matrix of size $m \times n$ can be mapped element-wise to a vector of size $m \times n$

$$V_{index} = M_{\lfloor \frac{index}{m} \rfloor, index \bmod n} \quad (5.4)$$

This shows that there exists a set of matrices $\{E_{ij} = |i\rangle\langle j|\}$ (the matrix units, in which exactly one element is 1 and every other element is 0) whose linear combination will span over every possible matrix of that particular dimension.

According to the Hilbert-Schmidt theorem, an arbitrary matrix of size $m \times n$ can be constructed from a linear combination of $m \times n$ matrix basis.

5.4 In-memory representation

Different types of errors are represented as Kraus operators.

5.4.1 Coherent and loss operators

Coherent Transformation

$$K_0 = \begin{pmatrix} \sqrt{1-\gamma}U_{HH} & \sqrt{1-\gamma}U_{HV} & 0 \\ \sqrt{1-\gamma}U_{VH} & \sqrt{1-\gamma}U_{VV} & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (5.5)$$

Loss Operator (H basis)

$$K_2 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ \sqrt{\gamma} & 0 & 0 \end{pmatrix} \quad (5.6)$$

Loss operator (V basis)

$$K_1 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & \sqrt{\gamma} & 0 \end{pmatrix} \quad (5.7)$$

5.4.2 Incoherent Error Operators

$$E_0 = \begin{pmatrix} \sqrt{1-p} & 0 & 0 \\ 0 & \sqrt{1-p} & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (5.8)$$

$$E_1 = \begin{pmatrix} \sqrt{p} & 0 & 0 \\ 0 & \sqrt{p} & 0 \\ 0 & 0 & 0 \end{pmatrix} \quad (5.9)$$

$$E_0 = \begin{pmatrix} \sigma_1 & \sigma_2 & 0 \\ \sigma_3 & \sigma_4 & 0 \\ L_H & L_V & I_{vac} \end{pmatrix} \quad (5.10)$$

(1) Superoperators

$$\{I, \sigma_X, \sigma_Y, \sigma_Z\} \oplus 0_{1 \times 1} \quad (5.11)$$

$$\mathcal{O}_{L_H} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix} \quad (5.12)$$

$$\mathcal{O}_{L_V} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \quad (5.13)$$

$$\mathcal{O}_{I_{vac}} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (5.14)$$

$$(5.15)$$

$$E_0 = \begin{pmatrix} \sigma_1 & \sigma_2 & 0 \\ \sigma_3 & \sigma_4 & 0 \\ L_H & L_V & I_{vac} \end{pmatrix} \quad (5.16)$$

Denoting by $\{\mathcal{O}_m\}_{m=1}^7$ the seven basis operators above, namely the four embedded Pauli operators $\{I, \sigma_X, \sigma_Y, \sigma_Z\} \oplus 0_{1 \times 1}$ together with $\mathcal{O}_{L_H}$, $\mathcal{O}_{L_V}$, and $\mathcal{O}_{I_{vac}}$, the channel of a linear section is fully specified by its 7×7 process matrix χ :

$$\mathcal{E}(\rho) = \sum_{m,n=1}^7 \chi_{mn} \mathcal{O}_m \rho \mathcal{O}_n^\dagger \quad (5.17)$$

Since χ is Hermitian and positive semidefinite, diagonalizing it as $\chi = UDU^\dagger$ with eigenvalues d_k recovers the operator-sum form

$$\mathcal{E}(\rho) = \sum_{k=1}^7 A_k \rho A_k^\dagger, \quad A_k = \sqrt{d_k} \sum_{m=1}^7 U_{mk} \mathcal{O}_m \quad (5.18)$$

so that at most 7 Kraus operators are needed, matching the dimension of the operator subspace.

Chapter 6

Representation Optimization

In the previous chapters, we have seen that an arbitrary linear optical component could be expressed as polarization mode based transfer matrix. One of the disadvantages of using an S-matrix is that the state space tends to explode, as the formalized definition requires that tensor product between all possible states be taken. As the number of interacting photons increases scatter matrix representation forces to look through all combination of lower number states. However, in most of the cases, the scatter matrix tends to be sparse: especially in the case where the beamsplitter is assumed to be perfect. Furthermore, most input photonic states are disjoint, meaning that each quantum state only occupies one arm of the photonic component at a time.

6.1 Occupied-Mode Reduction

Naively, given two disjoint quantum states, the combined scatter matrix would need to be 16×16 , as a result of taking the tensor product between the original 4×4 matrices,

$$S_{AB} = S_A \otimes S_B. \quad (6.1)$$

However, here, we can take advantage of the spatial disjointness, by essentially ignoring the unoccupied modes. Formally, this corresponds to taking the partial trace over the unoccupied subsystems, which can be computed efficiently without forming the full tensor product [5].

The reduction can be stated at the level of the scattering relation itself. A linear component acts on the creation operators of its modes as

$$\hat{a}_{\text{out},i} = \sum_{j=1}^M S_{ij} \hat{a}_{\text{in},j}, \quad (6.2)$$

so a state that only occupies the subset $\mathcal{M}$ of the input modes is only acted upon by the columns of S indexed by $\mathcal{M}$. Writing $P_{\mathcal{M}}$ for the projector that selects the occupied columns, the effective transfer matrix is the rectangular block

$$\tilde{S} = SP_{\mathcal{M}} \in \mathbb{C}^{M \times |\mathcal{M}|}, \quad (6.3)$$

and the amplitude vector $c \in \mathbb{C}^{|\mathcal{M}|}$ of a single excitation over the occupied modes transforms as $c_{\text{out}} = \tilde{S}c$. For a mixed state, the same block acts on both sides of the density matrix,

$$\rho_{\text{out}} = \tilde{S}\rho_{\text{in}}\tilde{S}^\dagger. \quad (6.4)$$

Rows of $\tilde{S}$ that are identically zero correspond to exit modes that can never be populated by the given input, and can be dropped as well.

In the worst case, this will result in a 4×16 sparse S-matrix, and in the best case, the dimension will be 4×4 , given a perfect beam splitter: since a perfect beam splitter maps each occupied input mode onto a fixed pair of output modes, the surviving block stays square. In general, the cost of applying a component scales with the number of occupied modes of the island, not with the dimension of the full formal mode space, which is what keeps the islands of interaction small enough to be collapsed as independent, dedicated matrix problems.

A rigorous derivation of this reduction is beyond the scope of this work. For the purpose of the simulator, it is sufficient that the reduced matrix reproduces the action of the full tensor product on every occupied mode, which holds by construction: the omitted rows and columns act only on unoccupied modes whose amplitudes are zero.

6.2 The Representation as a Contract

Taken together, the labeled mode notation, the seven-element Kraus basis of Chapter 5, and the occupied-mode reduced scattering matrices constitute a compact, transferable, universal operator format. This format is deliberately shaped as a contract: everything an engine needs in order to reconstruct and collapse the quantum state of an island is contained in the interaction record itself and in the operator values it references, with no dependence on the simulator’s implementation language or memory layout. The receiving end of this contract has not been implemented in this work beyond the in-process worker pool. However, the sufficiency arguments developed in these chapters are precisely what make such an implementation, in another language or on another machine, as easy as possible.

Chapter 7

Architectural Overview

7.1 Concurrency Architecture

To achieve high throughput and low latency, nodes (components) within the simulator run on separate threads. They communicate by passing wavepackets through thread-safe channels and queues. Entanglement between wavepackets is maintained globally in a central `InteractionStore`.

7.1.1 Centralized State Storage and Locking Registry

The centralized memory architecture relies on two primary structures to manage state efficiently across threads:

- **InteractionStore:** A global database of interaction records (islands of interaction) implemented as a magic ring buffer (sliding window) containing `UnsafeCell<InteractionCell>`. State IDs are assigned as sequential `u32` indices. State access is performed by calculating `state_id (mod offset)`, which allows for fast sequential lookups and memory reuse without the overhead of individual cell mutexes.
- **CellStateRegistry:** Holds a sliding bitmap of cell states, packed as two bits per cell (`Free`, `Locked`, `Moved`, `Retired`) inside `u64` words. The registry follows the same wrapping and indexing growth scheme as the `InteractionStore`. Locking utilizes a direct mapping from `state_id - offset` to the bitmap.

To ensure thread safety without severe bottlenecks, the system employs a **Greedy Lock Acquisition Protocol**:

1. A node thread acquires the `Registry` mutex.
2. The thread examines the `state_id` of the wavepackets within its input buffers.
3. If a state has been merged away, the `state_id` carried by the wavepacket is first forwarded through the tombstone chain left behind by the merge.
4. If a `state_id` is already marked as locked in the bitmap, the thread parks on a condition variable and retries once the holding thread releases its locks.

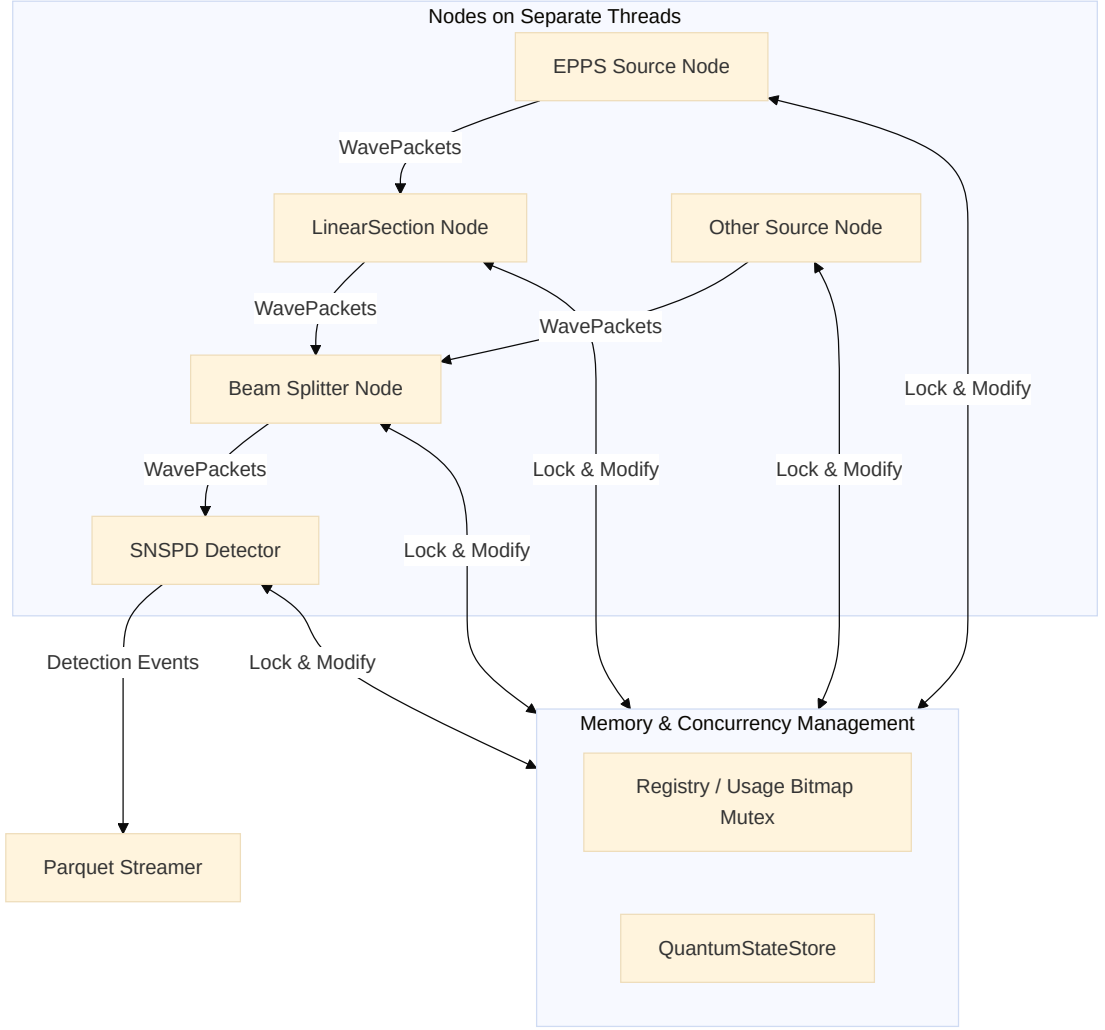


Figure 7.1: Overall multi-threaded architecture. Quantum state lives in the central store, which can be partially locked to enable concurrent processing

5. The thread marks all successfully scanned states as locked in the bitmap.
6. The **Registry** mutex is released.
7. The thread performs computations (via unsafe direct access to state data) on the locked states.
8. Upon completion, the thread re-acquires the **Registry** mutex, clears the locks for the processed state IDs, and releases the mutex.

Algorithm 1 lists this batch reservation protocol in full. The tombstone chains followed in step 3 are left behind by island merges; Algorithm 2 lists the merge procedure and its reference-count invariant.

Algorithm 1 Batch reservation on the interaction store. One mutex acquisition reserves every cell a batch needs; all state mutation then runs without the lock. **Invariant:** a cell is in state LOCKED if and only if exactly one slice holds it.

```

1: procedure ACQUIRESLICE(store, batch)
2:   lock store.mutex
3:   while true do                                     ▷ condvar wait loop
4:     blocked  $\leftarrow$  false
5:     for all wave packet  $w$  in batch do
6:       FOLLOWTOMBSTONES(store,  $w$ )                     ▷ Alg. 2, may rewrite  $w$ .handle
7:       if registry[ $w$ .handle] = LOCKED then
8:         blocked  $\leftarrow$  true; break
9:       end if
10:    end for
11:    if  $\neg$ blocked then break
12:    end if
13:    wait on store.condvar                               ▷ releases mutex while waiting
14:  end while
15:  for all distinct handle  $h$  referenced by batch do
16:    registry[ $h$ ]  $\leftarrow$  LOCKED
17:  end for
18:  unlock store.mutex
19:  ↩ slice over the reserved handles                     ▷ cells are now mutated lock-free
20: end procedure

21: procedure RELEASESLICE(slice)                         ▷ runs on slice drop
22:   lock store.mutex
23:   for all handle  $h$  in slice do registry[ $h$ ]  $\leftarrow$  FREE
24:   end for
25:   for all handle  $h$  that slice turned into a tombstone do
26:     if tombstone( $h$ ).refcount = 0 then registry[ $h$ ]  $\leftarrow$  RETIRED
27:     else registry[ $h$ ]  $\leftarrow$  MOVED
28:     end if
29:   end for
30:   pop RETIRED cells from the ring-buffer front
31:   unlock store.mutex; notify all waiters
32: end procedure

```

Algorithm 2 Island merge at a two-port component, with tombstone forwarding. **Invariant:** a tombstone’s reference count equals the number of wave packets still in flight that point to the old handle; the cell is retired exactly when the count reaches zero.

```

1: procedure MERGEISLANDS(slice, donor, recipient)
2:   offset  $\leftarrow$  recipient.modeMax ▷ shift donor modes into a fresh range
3:   for all operator  $op$  in donor.operators do
4:     append  $op$  with all mode indices + offset to recipient.operators
5:   end for
6:   for all (packet, mode) in donor.activePackets do
7:     recipient.activePackets.add(packet, mode + offset)
8:   end for
9:   recipient.modeMax  $\leftarrow$  recipient.modeMax + donor.modeMax
10:  recipient.terminatedCount  $\leftarrow$  recipient.terminatedCount +
    donor.terminatedCount
11:  cell(donor)  $\leftarrow$  TOMBSTONE(refcount = |donor.activePackets|, dest = recipient)
12: end procedure

13: procedure FOLLOWTOMBSTONES(store,  $w$ ) ▷ called under the store mutex
14:   while registry[ $w$ .handle] = MOVED do
15:      $t \leftarrow$  tombstone at  $w$ .handle
16:      $t$ .refcount  $\leftarrow$   $t$ .refcount - 1
17:     if  $t$ .refcount = 0 then
18:       registry[ $w$ .handle]  $\leftarrow$  RETIRED; drop the cell
19:     end if
20:      $w$ .handle  $\leftarrow$   $t$ .dest ▷ the destination may itself have moved
21:   end while
22: end procedure

```

7.1.2 Node Thread Lifecycle and Message Queues

Every node executes a continuous loop within its dedicated thread, following a strict lifecycle:

1. **Acquire State Locks:** Loop through input buffers, acquire state locks in the registry, and collect the wavepackets ready for processing.
2. **Process Batch:** Merge the islands of temporally overlapping wavepackets and append the corresponding operator records. No quantum state is evaluated at this stage.
3. **Release Locks:** Revert the corresponding lock bits in the registry.
4. **Push Outputs:** Transmit new wavepackets to downstream nodes in batches through bounded `mpsc` channels. The bounded capacity provides natural backpressure, keeping a fast producer at most a few batch windows ahead of its consumers.
5. **Idle/Wait:** If no input is pending, the thread parks on the blocking receive of its input channel, so an idle node consumes no CPU time.

Because `WavePacket` instances are owned by their respective threads, no additional locks are required when passing them between nodes. Figure 7.2 shows this thread and channel model.

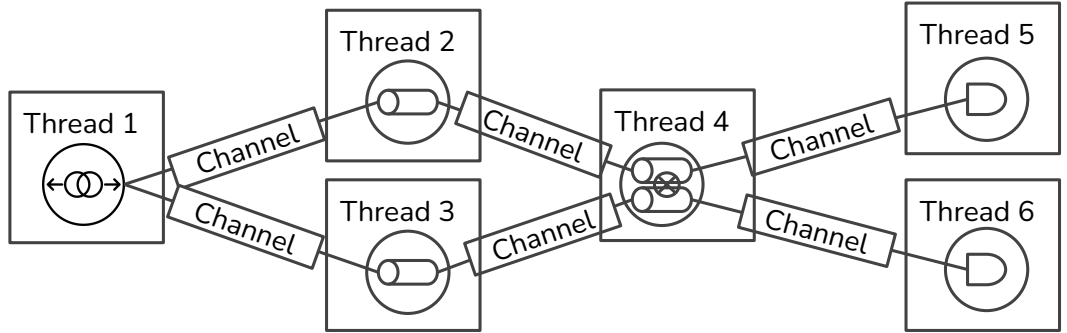


Figure 7.2: Thread and channel model of the simulator. Each node runs on its own thread, and wavepacket batches flow downstream through bounded channels.

7.2 Island Extraction and Delegated Collapse

The central observation behind this architecture is that a photonic testbed naturally decomposes into a large number of small, self-contained interaction records. Most wavepackets never interact with each other, and those that do form clusters of at most a few packets. In order to take advantage of this, the simulator does not evaluate any quantum state while the packets are in flight. Instead, each component simply appends an operator record to the `IslandOfInteraction` that the wavepacket belongs to, and forwards the packet downstream. The island is therefore a compact log of every interaction that

a cluster of entangled wavepackets has undergone, together with the identifiers needed to look up the actual operator values.

The collapse of an island is triggered by the detector that extracts the last remaining packet of that island. This is the same observation that underpins eventually consistent distributed systems [17]: operations on disjoint islands commute, so they can be executed out of order and in parallel without changing the final result. Since no other thread will ever touch a completed island again, the collapse requires no further synchronization: the islands of a batch are handed to a worker pool (implemented with rayon) as an embarrassingly parallel map, and no node ever waits on the progress of another node. The same property extends beyond a single machine. Because an island is self-contained, it can be serialized and shipped to a dedicated matrix engine on another computer, and the testbed simulation does not need to wait for the result. This effectively transforms the optical testbed simulation into a high-throughput generator of small, independent linear algebra problems, which can be distributed in a highly parallel manner.

Figure 7.3 shows this pipeline: the concurrency engine emits completed islands into the thread-pool quantum engine, whose results are written out as Parquet. Algorithm 3 lists the termination protocol that decides when an island is ready to collapse, and Algorithm 4 lists the Monte-Carlo walk that then evaluates it.

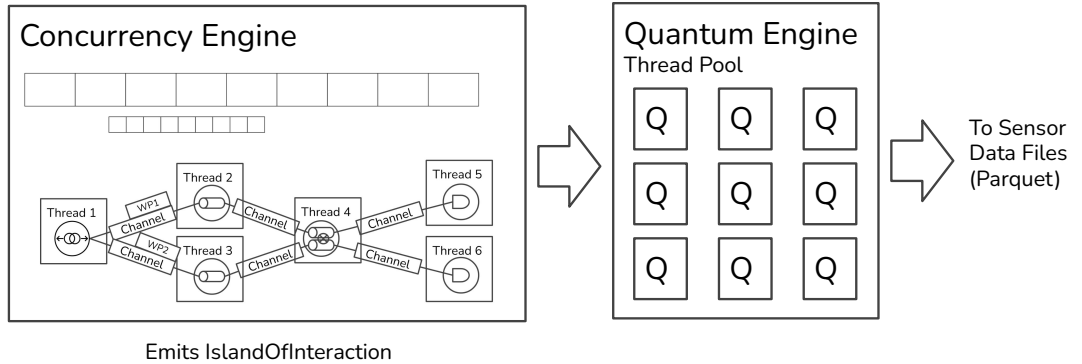


Figure 7.3: Delegated collapse pipeline. The concurrency engine hands completed islands to the rayon thread pool, and the collapse results are written to Parquet files.

Operator values themselves are kept in a separate operator store, and are referenced from the island by a store handle and a time stamp. This indirection keeps the islands small, and it is also what makes the simulation interactive: the operators of a component can be swapped at runtime through an atomically swapped pointer, without stopping the simulation, which models the time-varying behavior of a real testbed such as polarization drift and its compensation. The island, together with the operator values it references, thus constitutes a compact, transferable operator format: a contract between the testbed simulation and whatever engine evaluates it.

Algorithm 3 Detector-side termination protocol. A detector never removes wave packets from an island: removal would break a later merge. It only counts terminations. **Invariant:** an island is collapse-ready if and only if its terminated count equals its active-packet count.

```

1: procedure DETECTORPROCESSBATCH(spd, batch)
2:   slice  $\leftarrow$  ACQUIRESLICE(store, batch)
3:   for all wave packet  $w$  in batch do
4:     island  $\leftarrow$  slice[ $w$ .handle]
5:     island.terminatedCount  $\leftarrow$  island.terminatedCount + 1
6:     island.operators.push(SPD(spd.id,  $w$ .time, mode of  $w$ ))
7:   end for
8:   ready  $\leftarrow$  {  $h \in$  slice : island( $h$ ).terminatedCount = |island( $h$ ).activePackets| }
9:   snapshots  $\leftarrow$  copies of the islands in ready
10:  RELEASESLICE(slice) ▷ never hold a slice across a blocking call
11:  slice  $\leftarrow$  ACQUIRESLICEBYHANDLES(store, ready)
12:  for all ( $h$ , island) in ready  $\times$  snapshots in parallel do
13:    slice[ $h$ ]  $\leftarrow$  RESULT(COLLAPSE(island)) ▷ Alg. 4
14:  end for
15:  RELEASESLICE(slice)
16:  send batch to the claimer thread ▷ claimer waits for RESULT cells
17: end procedure

```

7.3 Physics Architecture and Representations

7.3.1 Wavepacket and Island Mechanics

The physical simulation relies on separating the spacetime metadata of a photon pulse from its underlying quantum state.

A **WavePacket** represents a single photon pulse. It carries its central time t (in picoseconds), its temporal spread σ_t , its wavelength λ and wavelength spread σ_λ , the handle of the island it belongs to, and a unique snowflake ID for identification. Time tags are utilized strictly for gating and temporal overlap checks, and are **not** used to encode quantum phase, which would require attosecond-level resolution. This in turn sacrifices the ability to model the global phase needed to simulate the Mach Zehnder Interferometer, but their collective behavior can be represented as a two-port linear optical component.

The leading and trailing edges of a packet are defined at three standard deviations (3σ representation). This effectively filters the simulated time window to a reasonable range, without needing to simulate the global hilbert space $\mathcal{H}_{global}$. Listing 7.1 shows the struct and its 3σ edge helpers.

Algorithm 4 Monte-Carlo collapse of one island. Phase 1 translates the recorded operator list into a consumer map; phase 2 pushes photons through it. $\mathcal{I}(w_1, w_2)$ is the pairwise indistinguishability of (7.10).

```

1: procedure COLLAPSE(island)
2:   consumer  $\leftarrow$  map from mode index to operator node
3:   for all operator  $op$  in island.operators, in record order do ▷ phase 1
4:     create the node for  $op$ ; register it as consumer of  $op$ 's source modes
5:     two-port interfering nodes whose source modes already belong to an
      earlier interfering node coalesce with it into one multi-mode router
      (union-find over operator handles)
6:   end for
7:   work  $\leftarrow$  [(sink mode, SINGLE) for each EPPS emission]
8:   while work is not empty do ▷ phase 2
9:      $(m, p) \leftarrow$  work.pop() ▷  $p \in \{\text{SINGLE}, \text{VAC}\}$ 
10:    switch consumer[ $m$ ]
11:      case KRAUS node with sink  $s$ : work.push( $((s, p))$ )
12:      end case
13:      case non-interfering splitter with sinks  $s_0, s_1$ :
14:        route  $p$  to  $s_0$  or  $s_1$  with probability  $\frac{1}{2}$ ; the other sink gets VAC
15:      end case
16:      case interfering splitter with sources  $w_1, w_2$ , sinks  $s_{0..3}$ :
17:        store  $p$ ; continue unless both source photons have arrived
18:        if both are SINGLE and coin( $\mathcal{I}(w_1, w_2)$ ) then
19:          both photons exit the same side:  $(s_0, s_2)$  or  $(s_1, s_3)$ , coin( $\frac{1}{2}$ )
20:        else
21:          route each photon independently, coin( $\frac{1}{2}$ ) each
22:        end if
23:      end case
24:      case multi-mode router:
25:        store  $p$ ; once every source mode has its photon, route each independently
26:      end case
27:      case SPD node:
28:        emit SUCCESS if  $p = \text{SINGLE}$ , else EMPTY
29:      end case
30:      case DUMP node: discard  $p$ 
31:      end case
32:    end switch
33:  end while
34:   emitted results ▷ one entry per terminated wave packet
35: end procedure

```

Listing 7.1: The WavePacket struct and its edge helpers

```

1 pub struct WavePacket {
2     pub time: Time,          // ps
3     pub time_sigma: u32,     // ps, single std dev.
4     pub wavelength: f32,
5     pub wavelength_sigma: f32,
6     pub state_handle: u32,
7     pub snowflake: u32,
8 }
9
10 impl WavePacket {
11     pub fn leading_edge(&self) -> u64 {
12         // three sigma
13         return self.time.saturating_sub(self.time_sigma as u64 * 3)
14         ;
15     }
16     pub fn trailing_edge(&self) -> u64 {
17         return self.time + self.time_sigma as u64 * 3;
18     }
19     pub fn overlaps(&self, wp: &WavePacket) -> bool {
20         return self.leading_edge() < wp.trailing_edge()
21             && wp.leading_edge() < self.trailing_edge();
22     }
23 }

```

In two-port components, two packets are considered to interact only if their 3σ envelopes overlap.

Within every stream, packets are kept sorted by their leading edge, and a batch collects the packets of one window of duration T (or at most N packets, whichever is reached first), so that a node can process an entire batch with a single acquisition of the store mutex.

(1) IslandOfInteraction

An `IslandOfInteraction` is the interaction record shared by a cluster of entangled wavepackets.

- **operators:** An append-only data store that stores the operators.
- **active_packets:** In-flight wave packets.
- **mode_max:** Counts decommissioned wave packets. Serves to keep track of the next mode index virtually.

Operator records never reference each other directly. instead, each record lists the virtual mode indices it consumes (source modes) and produces (sink modes), so the log forms a directed acyclic graph over the mode indices. The quantum state itself is materialized only at collapse, as a density matrix ρ (under the FSDM feature) or a state vector $|\psi\rangle$ (under FSSV). Figure 7.4 shows how this operator log reads as a quantum circuit, and Listing 7.2 shows the struct and its collapse-readiness check.

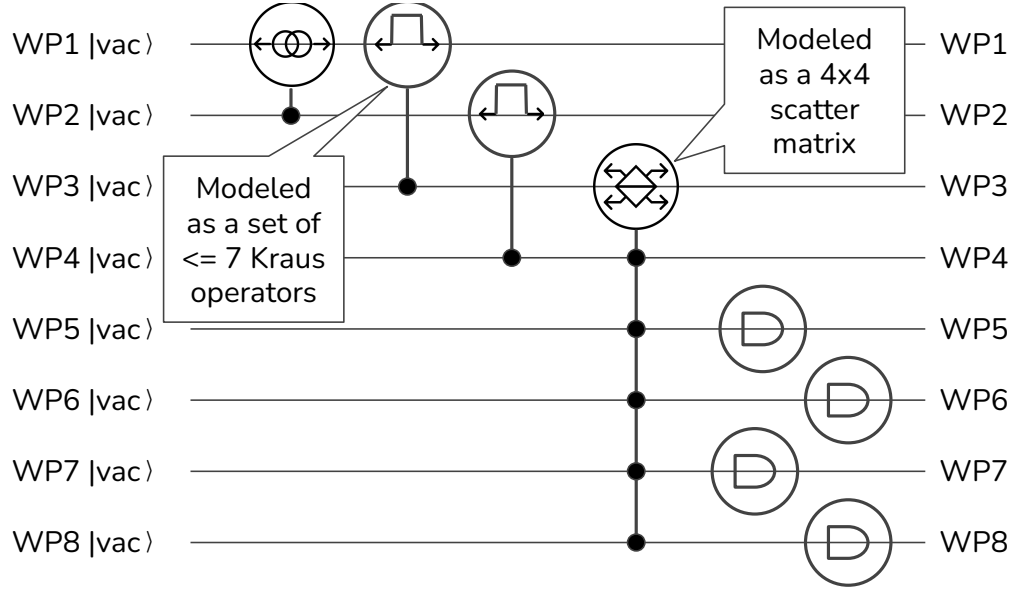


Figure 7.4: Circuit interpretation of an island. The operator DAG over the virtual mode indices reads as a qutrit circuit over $|H\rangle$, $|V\rangle$, and $|\text{vac}\rangle$.

Listing 7.2: The IslandOfInteraction struct and its readiness check

```

1 // parameter tuned to be packed in 512 bytes
2 pub struct IslandOfInteraction {
3     pub operators: SmallVec<[Operator; 13]>,
4     active_packets: ActivePacketStore,
5     pub mode_max: u16, // mode_index < mode_max
6     pub terminated_packet_count: u8,
7 }
8
9 impl IslandOfInteraction {
10     pub fn register_wavepacket(&mut self, wp: &WavePacket) ->
        ModeIndex {
11         let mode_index = self.mode_max;
12         self.active_packets.add(wp.snowflake, self.mode_max);
13         self.mode_max += 1;
14         mode_index
15     }
16     pub fn is_collapse_ready(&self) -> bool {
17         self.active_packets.len() == self.terminated_packet_count
18     }
19 }

```

- *Merging Mechanics:* When islands merge (e.g., at a Beam Splitter), the donor islands are drained into the recipient: the donor's records are appended with their mode indices relabeled by the offset of the recipient's mode counter,

$$m_{\text{donor}} \mapsto m_{\text{donor}} + m_{\text{recipient}}, \quad (7.1)$$

and each donor slot is replaced with a reference-counted tombstone that forwards late-arriving wavepackets to the merged island. The reference count is initialized to the number of packets that will still look the donor handle up. Every forwarding decrements it, and the last one retires the slot. Listing 7.3 shows this merge and the tombstone it leaves behind.

- *Life Cycle*: Islands are never split. Once created, they either merge (tombstoning the donor state ID) or are collapsed into a measurement result and retired once their last referencing wavepacket is detected ($A = \emptyset$), returning the slot to the ring buffer.

Listing 7.3: Island merge and tombstone creation in the interaction store

```

1 pub struct Tombstone {
2     ref_cnt: u8,
3     move_destination: u32,
4 }
5
6 // NOTE: both donor and recipient are assumed to be
   IslandOfInteraction
7 pub fn merge_islands(&mut self, donor_handle: u32, recipient_handle
   : u32) {
8     // NOTE: donor_handle and recipient_handle are provably
       disjoint,
9     // so we get multiple mutable borrows using unsafe
10    let donor_cell = unsafe { self.unsafely_get_mut(donor_handle)
       };
11    let donor = donor_cell.unwrap_as_island_or_else(|_| {
12        panic!("Merge failed. Donor is not IslandOfInteraction");
13    });
14    let recipient = unsafe { self.unsafely_get_mut(recipient_handle
       ) }
15        .unwrap_as_island_or_else(|_| {
16        panic!("Merge failed. Recipient is not
           IslandOfInteraction");
17    });
18    let offset = recipient.mode_max;
19    for operator in donor.operators.iter() {
20        recipient.operators.push(operator.clone_with_offset(offset)
       );
21    }
22    for (packet_id, mode_idx) in donor.active_packets.
       active_packets.iter() {
23        recipient.active_packets.add(*packet_id, mode_idx + offset)
       ;
24    }
25    recipient.mode_max += donor.mode_max;
26    recipient.terminated_packet_count += donor.
       terminated_packet_count;
27    *donor_cell = InteractionCell::Tombstone(Tombstone {
28        ref_cnt: donor.active_packets.len(),
29        move_destination: recipient_handle,

```

```

30     });
31     self.moved.push(donor_handle);
32 }

```

7.3.2 Component Models

The simulator models the optical components of the testbed by mapping their physical behaviors to discrete operations on the `WavePacket` and `IslandOfInteraction` structures.

- **Entangled Photon Pair Source (EPPS):** Generates entangled wavepacket pairs on a pulsed pump grid. Instead of rolling a die at every pump pulse, the number of pulses until the next successful pair generation is sampled directly from the geometric distribution. For each pair, a fresh island is created whose EPPS record references the 4×4 density matrix obtained from the tomography of the PPLN crystal output, over the modes $|H_s\rangle, |V_s\rangle, |H_i\rangle, |V_i\rangle$ of the signal and idler arms. Algorithm 5 lists this lazy sampling loop.
- **Linear Section:** Models components such as wave plates and fiber segments, including attenuation and decoherence. Each traversal appends a record that references the seven Kraus operators of the section derived in Chapter 5, and shifts the packet time by the configured propagation delay.
- **Dual-Port Components (BS / PBS):** The evaluation of a dual-port component is performed in two discrete steps: Calculation of temporal overlap, and the calculation of the overlap of the underlying polarization quantum states. Since the main goal of this work is to reduce the testbed states into computable operator expression, only the first step is performed by the concurrency engine. In Hong-Ou-Mandel interference, indistinguishability corresponds to the inner product of the participating states.

$$\mathcal{I}(w_1, w_2) = |\langle w_1 | w_2 \rangle|^2 \quad (7.2)$$

Since time and polarization are independent degree of freedom, they form orthogonal basis in bosonic hilbert space $\mathcal{H}_{w_1, w_2}$. Since temporal basis and the polarization basis are orthogonal, they can be calculated individually, and combined later to give the final indistinguishability.

$$\mathcal{I}(w_1, w_2) = |\langle w_{1,t} | w_{2,t} \rangle|^2 |\langle w_{1,\text{pol}} | w_{2,\text{pol}} \rangle|^2 \quad (7.3)$$

In the concurrency engine, we solve the first half of this equation. The temporal distribution function. First part of this indistinguishability could be evaluated using the gaussian wave distribution function, defined inside the `WavePacket` struct.

$$w(t) = \frac{1}{\sqrt{2\pi\sigma_w^2}} \exp\left(-\frac{(t - t_w)^2}{2\sigma_w^2}\right) \quad (7.4)$$

Since this indicates the temporal probability distribution of the wave function, in order to get the probability amplitude, we need to take a square root

$$\sqrt{w(t)} = \frac{1}{(2\pi\sigma_w^2)^{1/4}} \exp\left(-\frac{(t - t_w)^2}{4\sigma_w^2}\right) \quad (7.5)$$

In order to account for the chromatic mode, we multiply this wave envelope with the phase term

$$a(t) = \sqrt{w(t)} e^{-i\omega_w t} = \frac{1}{(2\pi\sigma_w^2)^{1/4}} \exp\left(-\frac{(t-t_w)^2}{4\sigma_w^2} - i\omega_w t\right) \quad (7.6)$$

The inner product between two such wave functions can be calculated with the following integral

$$\langle w_{1,t} | w_{2,t} \rangle = \int_{-\infty}^{\infty} a_1^*(t) a_2(t) dt \quad (7.7)$$

$$= \frac{1}{\sqrt{2\pi\sigma_1\sigma_2}} \int_{-\infty}^{\infty} \exp\left(-\frac{(t-t_1)^2}{4\sigma_1^2} - \frac{(t-t_2)^2}{4\sigma_2^2} + i(\omega_1 - \omega_2)t\right) dt \quad (7.8)$$

$$= \frac{1}{\sqrt{2\pi\sigma_1\sigma_2}} \exp\left(\frac{\Delta t^2}{4(\sigma_1^2 + \sigma_2^2)} - \frac{\sigma_1^2 \sigma_2^2 \Delta \omega^2}{\sigma_1^2 + \sigma_2^2}\right) \quad (7.9)$$

Finally, squaring this function gives the indistinguishability in terms of time envelop and frequency.

$$\mathcal{I}(w_1, w_2) = \frac{2\sigma_1\sigma_2}{\sigma_1^2 + \sigma_2^2} \exp\left(-\frac{\Delta t^2}{2(\sigma_1^2 + \sigma_2^2)} - \frac{2\sigma_1^2\sigma_2^2 \Delta \omega^2}{\sigma_1^2 + \sigma_2^2}\right), \quad (7.10)$$

The code for this calculation is given in Listing 7.4.

Listing 7.4: Temporal indistinguishability of two wave packets

```

1  pub fn indistinguishability(&self, other: &WavePacket) ->
    f64 {
2      let s1 = self.time_sigma as f64;
3      let s2 = other.time_sigma as f64;
4
5      // Degenerate (delta-like) packets: identical carriers
        & times -> 1, else 0.
6      if s1 == 0.0 || s2 == 0.0 {
7          return if self.time == other.time
8              && self.wavelength == other.wavelength
9              && s1 == s2 { 1.0 } else { 0.0 };
10     }
11
12     // Delta t in ps - go through i128 first: u64 -> f64
        loses integer
13     // precision above 2^53, and the *difference* is what
        matters.
14     let dt = (self.time as i128 - other.time as i128) as
        f64;
15
16     // carrier angular frequency detuning, rad/ps
17     let w1 = 2.0 * PI * C_NM_PER_PS / self.wavelength as
        f64;
18     let w2 = 2.0 * PI * C_NM_PER_PS / other.wavelength as
        f64;
19     let dw = w1 - w2;

```

```

20
21     let ssum = s1 * s1 + s2 * s2;
22     let prefactor = 2.0 * s1 * s2 / ssum;
23     let exponent = -dt * dt / (2.0 * ssum)
24                   - 2.0 * (s1 * s1) * (s2 * s2) * dw * dw
25                     / ssum;
26
27     prefactor * exponent.exp()
    }

```

The island of interaction are then merged with this indistinguishability value.

- **Superconducting Nanowire Single-Photon Detector (SNSPD):** The terminal component. It extracts the arriving packet from its island's registry and appends a detection record with the time tag. The detector that extracts the last packet of an island owns its collapse: the island is evaluated into detection events, and the slot is retired. The resulting time tags are emitted to the consumer of the simulation. Algorithm 6 lists the classical detector model applied to the claimed click stream, and Listing 7.5 shows the claimer loop that implements it.

Algorithm 5 Lazy pair generation at the EPPS. Instead of testing every pump pulse (Poisson process, one wake-up per pulse), the source samples the number of pulses until the next success from the geometric distribution. Both formulations produce the same statistics; the lazy one costs one sample per emitted pair.

```

1: procedure GENERATEBATCH(epps, constraint)
2:   while epps.time ≤ constraint.timeout and |pairs| < constraint.maxSize do
3:      $u \leftarrow \text{uniform}(0, 1)$ 
4:      $k \leftarrow \lfloor \ln u / \ln(1 - p_{\text{success}}) \rfloor$  ▷ pulses until the next success
5:     epps.time ← epps.time +  $k \cdot T_{\text{pump}}$  ▷ fractional picoseconds carried in an
        accumulator
6:     emit signal and idler wave packets stamped with epps.time
7:     create one island per pair, register both packets, record the EPPS
        operator with the pair's density matrix
8:   end while
9: end procedure

```

Listing 7.5: The SPD claimer thread loop

```

1 thread::spawn(move || {
2     let mut last_click: Time = 0;
3     loop {
4         let mut batch = rx.recv().unwrap();
5         let mut result = ctx.interaction_store
6             .claim_collapsed_packets(&mut batch.batch);
7         result.sort_unstable();
8         // apply the dead time, dark count, and efficiency
9         let mut filtered_result: Vec<u64> = Vec::new();
10        for time in result.iter() {
11            if rand::random_bool(1.0 - efficiency) {

```

```

12         continue;
13     }
14     if last_click + dead_time > *time {
15         continue;
16     }
17     let time_delta = (*time).saturating_sub(last_click);
18     if rand::random_bool(time_delta as f64 / (1.0e12 /
19         dark_count)) {
20         filtered_result.push(rand::random_range(last_click
21             ..*time));
22     }
23     filtered_result.push(*time);
24     last_click = *time;
25 }
26 // ... forward filtered_result to the time tagger
27 })

```

7.3.3 Temporal disjointness

Another important constraint in this simulation architecture was to warrant the temporal and spatial disjointness of the wavepacket. In a typical SPDC-based testbed, about 10^7 photon pairs are generated each second. However, most of those photons do not interact with each other, resulting in a low thousands of interfered photons per second. In order to resolve this, temporal disjointness of each wavepacket should be warranted, unless those that are close to each other interact inside of a beamsplitter. In order to achieve this, temporally disjoint sets of wavepackets are handled as WavePacket structures. This ensures that even given a large number of photons inside the simulated environment, the amount of information that is exchanged between those disjoint spaces of memory remains minimal. Furthermore, quantum entanglement information of each set of interacting wavepackets could be stored in a specialized data structure that ensures the disjointness of the data, and this effectively transforms the photonic optical testbed simulation into thousands of parallel instances of quantum computer with approximate qubit count of 2 to 10. This enables, with the right combination of data structure and algorithm, a massive parallel scaleup of the simulation.

7.4 Configuration Ergonomics

A simulator is only useful if a new experiment is cheap to write. The public API of the simulator is therefore built around two ideas: a cloneable simulation context and the builder pattern.

The context object owns every shared resource: the interaction store, the configuration, the operator record, and the telemetry registry. It is a small handle that can be cloned freely, because each field is behind an atomically reference-counted pointer. Every node receives a clone of the same context, so wiring a topology never requires global variables.

Each component type provides a builder that starts from physically sensible defaults. The user only sets the parameters that differ from the defaults, then calls `spawn()`, which starts the node on its own thread and registers its telemetry slot. Listing 7.6 shows the detector builder: a default detector is ideal except for the measured properties of the laboratory hardware (90% efficiency, 10^2 dark counts per second, 20 ns dead time), and every property can be overridden with one chained call.

Listing 7.6: The SPD builder. Defaults model the laboratory detector; each setter overrides one parameter.

```
1 let spd_1 = ctx.spd().id(1).spawn();
2 let spd_2 = ctx.spd().id(2)
3   .dead_time(20_000)    // ps
4   .dark_count(100.0)    // counts per second
5   .efficiency(0.9)
6   .spawn();
```

Topology construction is a chain of `connect` calls between exit ports and entry ports, as shown in Listing 8.2. The builder returns a handle, the handle exposes its ports, and the type system rejects a connection to a port that does not exist. As a result, the full Hong–Ou–Mandel experiment of Chapter 8, including its parquet output and its throughput monitor, is one short function, and a new topology is a matter of minutes rather than a new program.

This ergonomic layer is thin: the builders only fill template structs and hand them to the same spawn machinery that the rest of this chapter described. It adds no overhead on the hot path.

Algorithm 6 Classical detector model, applied by the claimer thread to the claimed click stream. Detection times are sampled from each packet’s Gaussian envelope, so the claimed stream is only near-sorted and is sorted first. lastClick persists across batches. Defaults: $\eta = 0.9$, $r_{\text{dark}} = 100 \text{ s}^{-1}$, $\tau_{\text{dead}} = 20 \text{ ns}$ (non-paralyzable).

```

1: procedure APPLYDETECTORMODEL(clicks, lastClick)
2:   sort clicks ascending                                ▷ adaptive sort; input is near-sorted
3:   out  $\leftarrow$  [ ]
4:   for all  $t$  in clicks do
5:     if  $\neg \text{coin}(\eta)$  then continue
6:     end if                                              ▷ detection efficiency
7:     if lastClick +  $\tau_{\text{dead}} > t$  then continue
8:     end if                                              ▷ dead time
9:      $\Delta \leftarrow t - \text{lastClick}$ 
10:    if  $\text{coin}(\Delta \cdot r_{\text{dark}})$  then                        ▷ dark count in the idle gap
11:      out.push(uniform(lastClick,  $t$ ))
12:    end if
13:    out.push( $t$ ); lastClick  $\leftarrow t$ 
14:  end for
15:   out                ▷ tagged batch also carries the time span for the merger
16: end procedure

```

Chapter 8

Experiment Setup

In order to evaluate the validity as well as the throughput of the simulation software, an experimental setup demonstrating the Hong-Ou-Mandel interference [24] has been implemented.

The simulation parameters follow the testbed described in the background chapter. Both sources run on a 1 GHz pump grid with a pair generation probability of 1% per pulse, corresponding to approximately 10^7 pairs per second per source. Each wavepacket is generated with a temporal spread of $\sigma_t = 75$ ps at a wavelength of 1550 nm. Wavepackets are streamed between the nodes in batches bounded by a window of 200 μ s of simulated time or 2000 packets, whichever is reached first. The eight component nodes each run on their own thread, and the collapse of completed islands is handed to a shared worker pool. After their states are collapsed through the Monte-Carlo process, the resulting states are claimed by the parquet file writer, and saved to a parquet file.

In order to ensure the repeatability of the experiment, an external rust script has been written to oversee the experiment, and to collect the data. The resulting data is compiled down to csv format, then it is passed down to dedicated python script to be rendered as a matplotlib diagram.

8.1 Hardware and Runtime

All experiments run on one workstation. The processor is an AMD Ryzen AI 9 HX PRO 375 with 24 logical cores. Each core has 80 KB of L1 cache and 1 MB of private L2 cache. All cores share a 24 MB L3 cache. The system has 64 GB of LPDDR5X memory. The operating system is Arch Linux (x86_64), kernel 7.1.4-arch1-1.

The cache hierarchy is important for the measured throughput. The full working set of the simulator has been measured to fit inside the L3 cache. The shared telemetry records and the hot per-node state fit inside the L1 cache of each core. Cross-core traffic therefore stays inside the cache hierarchy and does not reach main memory.

The simulator is compiled in release mode with `target-cpu = "native"`. This flag lets the compiler use the full instruction set of the host processor. The generated code uses 256-bit AVX vector instructions for the wavepacket mathematics.

8.2 Telemetry Context

In order to record the runtime behavior of the simulation, telemetry parameter is stored as `Arc` in the simulation context, which are then accessed by worker threads, and its atomic fields are updated.

The telemetry struct follows the Struct of Arrays (SoA) format, aligned to the cache line boundary with the means of `crossbeam_utils::CachePadded`.

Listing 8.1: Telemetry metric structs

```
1 pub struct BaseMetrics {
2     pub packet_consumed: AtomicU64,
3     pub packet_generated: AtomicU64,
4     pub sim_time: AtomicU64,
5 }
6
7 pub struct SPDMetrics {
8     pub base: BaseMetrics,
9     pub clicks: AtomicU64,
10    pub dark_counts: AtomicU64,
11 }
12
13 pub struct DualMetrics {
14     pub base: BaseMetrics,
15     pub merged_states: AtomicU64,
16     pub paired_packets: AtomicU64,
17     pub single_packets: AtomicU64,
18     pub multi_packet_event: AtomicU64,
19     pub pair_count: AtomicU64,
20 }
21
22 pub struct TelemetryRecord {
23     pub epps: boxcar::Vec<CachePadded<BaseMetrics>>,
24     pub single: boxcar::Vec<CachePadded<BaseMetrics>>,
25     pub dual: boxcar::Vec<CachePadded<DualMetrics>>,
26     pub spd: boxcar::Vec<CachePadded<SPDMetrics>>,
27 }
```

Each worker thread is assigned a `worker_id` based on the ordering within the telemetry record, which is shared between the worker handle struct, which resides on the main thread, and the worker struct, which resides on the worker thread. The worker thread then updates the relevant fields of the telemetry struct using atomic operations. The main thread then reads off the values, and writes down the summary to `stdout`, which is then read by the parent process, which oversees the entire experiment.

8.3 Topology

The topology consists of single EPPS, two optical delay lines, beam splitters, two SPDs, and a 30cm section of fiber leading to the SPD. The code entailing this setup is listed

below:

Listing 8.2: HOM Topology Setup

```
1 let ctx = SimulationContext::new().update_config(|config| {
2     config.batch.period = 200_000_000;
3     config.batch.max_size = 2000;
4 });
5 let epps = ctx.epps().spawn();
6
7 // free-space ODL: arm length -> propagation delay at vacuum c
8 let odl_1 = ctx.single().delay(
9     (odl_arm_1_m / 299_792_458.0 * 1.0e12).round() as u64
10 ).spawn();
11 let odl_2 = ctx.single().delay(
12     (odl_arm_2_m / 299_792_458.0 * 1.0e12).round() as u64
13 ).spawn();
14
15 let bs = ctx.dual().spawn();
16
17 let spd_fiber_1 = ctx.single().delay(
18     (0.3f64 / 299_792_458.0 * 1.0e12).round() as u64
19 ).spawn();
20 let spd_fiber_2 = ctx.single().delay(
21     (0.3f64 / 299_792_458.0 * 1.0e12).round() as u64
22 ).spawn();
23
24 let spd_1 = ctx.spd().id(1).spawn();
25 let spd_2 = ctx.spd().id(2).spawn();
26
27 epps.exit_port(0).connect(odl_1.entry_port(0));
28 odl_1.exit_port(0).connect(bs.entry_port(0));
29 epps.exit_port(1).connect(odl_2.entry_port(0));
30 odl_2.exit_port(0).connect(bs.entry_port(1));
31 bs.exit_port(0).connect(spd_fiber_1.entry_port(0));
32 bs.exit_port(1).connect(spd_fiber_2.entry_port(0));
33 spd_fiber_1.exit_port(0).connect(spd_1.entry_port(0));
34 spd_fiber_2.exit_port(0).connect(spd_2.entry_port(0));
```

Chapter 9

Experiment and Evaluation

9.1 Arm Sweep Experiment

In order to validate the output of the simulation, and to cross reference the result from real measurement in the laboratory, an arm-sweep experiment has been performed. One of the optical delay lines, demarcated `odl_1`, is fixed at $L_1 = 15.0$ cm, while the other, `odl_2`, is swept from 0 to 30.0 cm in increments of $\Delta s = 0.1$ cm. The path difference $\delta = L_2 - L_1$ therefore covers the symmetric range $[-15.0, +15.0]$ cm around the balanced point $\delta = 0$, at which the Hong–Ou–Mandel dip is expected. For each arm setting the simulator is executed as an independent process for a fixed wall-clock budget of 100 ms, and the coincidence-packet rate, normalised by the elapsed simulation time, is recorded. The procedure is summarised in Algorithm 7.

Algorithm 7 Arm-sweep experiment

```
1:  $L_1 \leftarrow 15.0$  cm
2: for  $L_2 \leftarrow 0$  to 30.0 cm step 0.1 cm do
3:    $\delta \leftarrow L_2 - L_1$ 
4:   run simulator with  $(L_1, L_2)$  for 100 ms
5:   record  $(\delta, L_2, \text{packet rate})$ 
6: end for
```

9.2 Arm Sweep Experiment – Result

The sweep was performed in two detector configurations: an ideal configuration with unit detection efficiency, no dark counts, and a dead time of 1 ns, and a realistic configuration with a detection efficiency of $\eta = 0.9$, a dark-count rate of 100 s^{-1} , and a dead time of 20 ns. The results are shown in Figure 9.1 and Figure 9.2.

The expected depth of the dip follows from the counting statistics of the detectors. Two photons that arrive at the same detector within its dead time produce a single click. For distinguishable photons the balanced beam splitter yields four equally likely outcomes: both photons at SPD 1, both at SPD 2, or one photon at each detector (two permutations). The first two outcomes produce one click, the latter two produce two

clicks, giving an expectation of 1.5 clicks per pair. For fully indistinguishable photons the pair always exits through a common port, producing exactly one click per pair. The click rate at the balanced point is therefore expected to drop to 2/3 of the baseline, a dip of approximately 33%.

Outcome	Probability	Photons per port (SPD 1, SPD 2)	Clicks
<i>Distinguishable photons</i> ($\delta \gg c\sigma_t$)			
Both to SPD 1	1/4	(2, 0)	1
Both to SPD 2	1/4	(0, 2)	1
Split, photon 1 up	1/4	(1, 1)	2
Split, photon 2 up	1/4	(1, 1)	2
Expected clicks per pair			1.5
<i>Indistinguishable photons</i> ($\delta = 0$)			
Both to SPD 1	1/2	(2, 0)	1
Both to SPD 2	1/2	(0, 2)	1
Split	0	(1, 1)	—
Expected clicks per pair			1.0
Click ratio (dip)			2/3

In the ideal configuration the recorded rate falls from a baseline of $1.50 \times 10^7 \text{ s}^{-1}$ to a minimum of $1.01 \times 10^7 \text{ s}^{-1}$, a ratio of 0.671. This matches the predicted value of 2/3 to within the statistical noise of the run and corresponds to full two-photon interference visibility.

In the realistic configuration the dip is shallower: the rate falls from $1.23 \times 10^7 \text{ s}^{-1}$ to $9.10 \times 10^6 \text{ s}^{-1}$, a ratio of 0.738. The reduction in contrast is caused by the detector model, through two mechanisms. First, the sub-unit efficiency affects bunched and split events asymmetrically: a split event yields $2\eta = 1.8$ expected clicks, while a bunched pair still produces a click with probability $1 - (1 - \eta)^2 = 0.99$, since the second photon acts as redundancy. Efficiency loss therefore suppresses the baseline more than the minimum, raising the click ratio to 0.710. Second, the 20 ns dead time removes a rate-dependent fraction of clicks; the loss is larger on the wings, where the rate is higher, which compresses the contrast further to the observed 0.738. The dark-count rate of 100 s^{-1} is eight orders of magnitude below the signal rate and has no measurable effect in this regime.

The temporal spread of $\sigma_t = 75 \text{ ps}$ corresponds to a spatial extent of $c\sigma_t \approx 2.25 \text{ cm}$ in free space. The width of the dip therefore matches the wavepacket overlap: the visibility falls off over a path difference of one spatial sigma, and interference is no longer observable beyond $3c\sigma_t \approx 6.75 \text{ cm}$, where the wavepackets no longer overlap. This is consistent with both recorded dips, which span roughly $\pm 7 \text{ cm}$ around the balanced point.

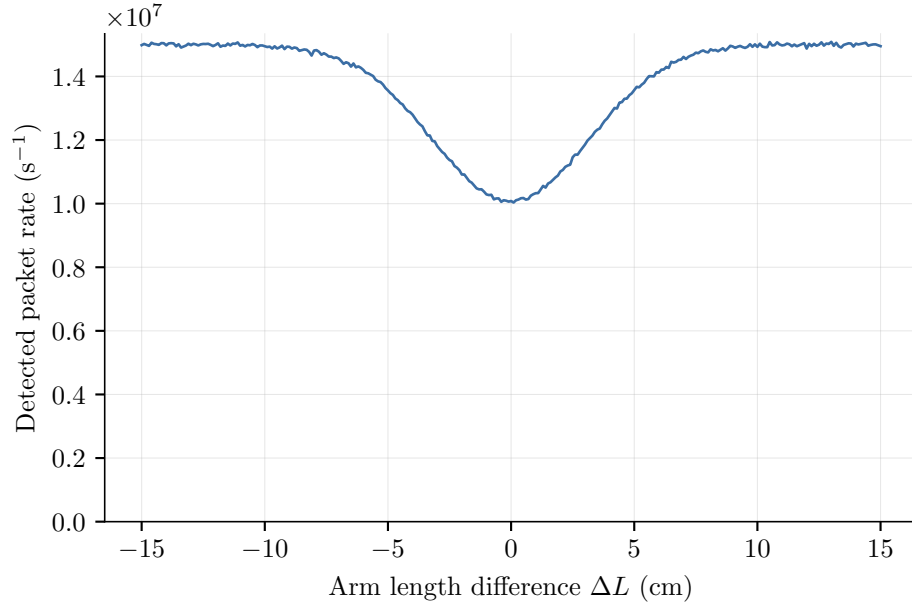


Figure 9.1: Detected packet rate as a function of the arm length difference, with ideal detectors ($\eta = 1$, no dark counts, 1 ns dead time).

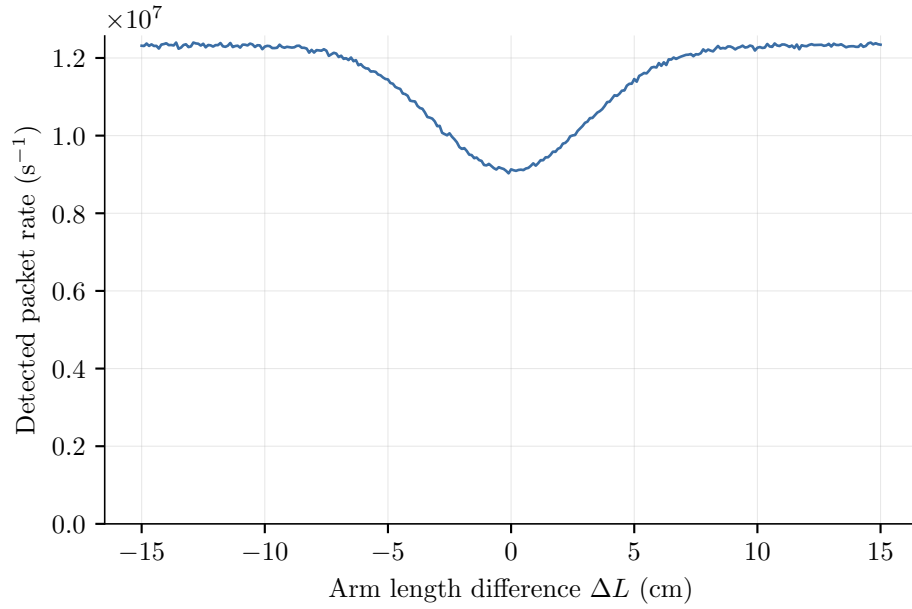


Figure 9.2: Detected packet rate as a function of the arm length difference, with realistic detectors ($\eta = 0.9$, 100 s^{-1} dark counts, 20 ns dead time).

9.3 Core Sweep Experiment

In order to assess the multithreading efficiency of the simulation architecture, the number of available cores was varied using the `taskset` command. The number of cores was varied from 1 to 24, with the arm lengths held fixed at the balanced setting. Cores are added in a fixed, topology-aware order, so that each measurement at n cores uses the first n entries of the same CPU list. For each core count the simulator is pinned to the selected CPUs and executed over a fixed measurement window, and the sustained event rate is recorded. The procedure is summarised in Algorithm 8.

As a point of reference, the throughput of QuISP [2] was measured on the same machine. The QuISP scenario uses the `Simple_MM` topology: two QNodes joined by a 1 km classical channel and a 1 km quantum channel, with no midpoint node, so that the Bell-state analyzer resides in the receiving QNIC. The traffic consists of link tomography: after a 1 s startup buffer, the nodes generate entangled pairs at up to 10^6 photons per second and exchange one classical measurement-result packet per pair, for 10^7 measurements in total. Every classical packet that crosses the wire is counted through the `txBytes/rxBytes` statistics of each node, and this count divided by the run time gives the packet rate. All error rates (channel, gate, memory, and measurement) are set to zero, so the quantum-state calculations of the GraphState backend are trivial and the packet flow is deterministic; QuISP offers no option to disable state tracking entirely. Under this favorable zero-noise configuration, QuISP sustains 1.12×10^7 packets per second on a single core. QuISP is single-threaded, so this rate does not change with the number of available cores; it is drawn as a horizontal reference line in Figure 9.3.

Algorithm 8 Core-sweep experiment

- 1: fix arm lengths (L_1, L_2)
 - 2: $C \leftarrow$ ordered list of CPUs
 - 3: **for** $n \leftarrow 1$ **to** $|C|$ **do**
 - 4: pin simulator to first n CPUs of C
 - 5: run for the measurement window
 - 6: record (n , event rate)
 - 7: **end for**
-

9.4 Memory Usage

The heap consumption of the simulator was recorded with `heaptrack` over a run of approximately 20 s. The topology is identical to that of Section 8.3. Two configurations were measured: one with the parquet file writer enabled, and one with the parquet file writer removed from the build. The recorded consumption is shown in Figure 9.4 and Figure 9.5.

With the parquet writer enabled, heap consumption rises to a plateau of approximately 250 MB after roughly 2 s and remains flat for the rest of the run. The largest allocation bands belong to the write buffers of the parquet writer. Short periodic dips in the top bands coincide with the flushing of completed row groups.

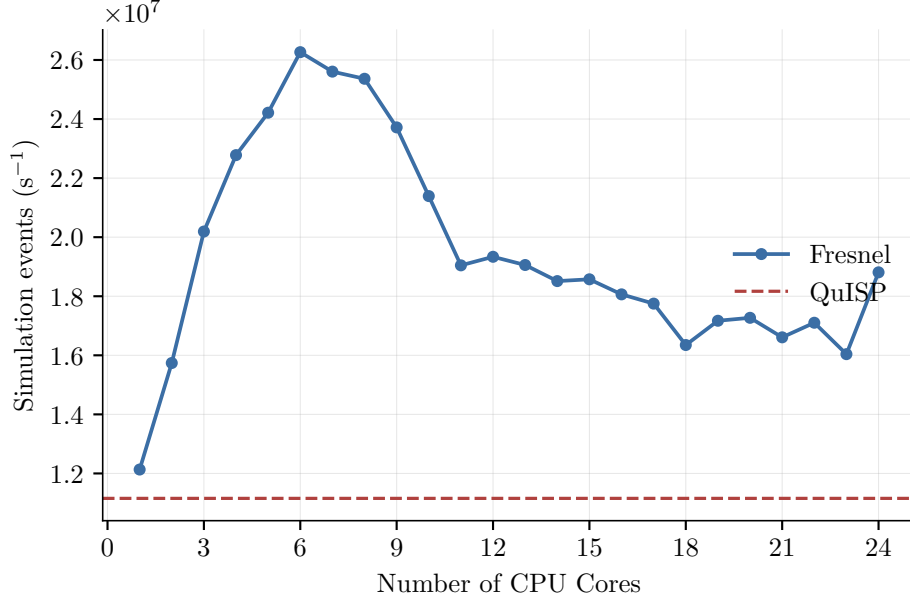


Figure 9.3: Sustained event rate as a function of core count, with realistic detectors ($\eta = 0.9$, 100 s^{-1} dark counts, 20 ns dead time). The dashed line marks the single-core QuISP baseline of 1.12×10^7 packets per second, measured in a zero-noise link-tomography scenario on the same machine.

With the parquet writer removed, heap consumption settles at approximately 47 MB after an initial phase of approximately 26 MB, and remains flat for the rest of the run. The dominant band corresponds to the pre-allocated buffers of the simulation core, including the interaction store and the per-node batch queues. In both configurations the consumption shows no growth over time.

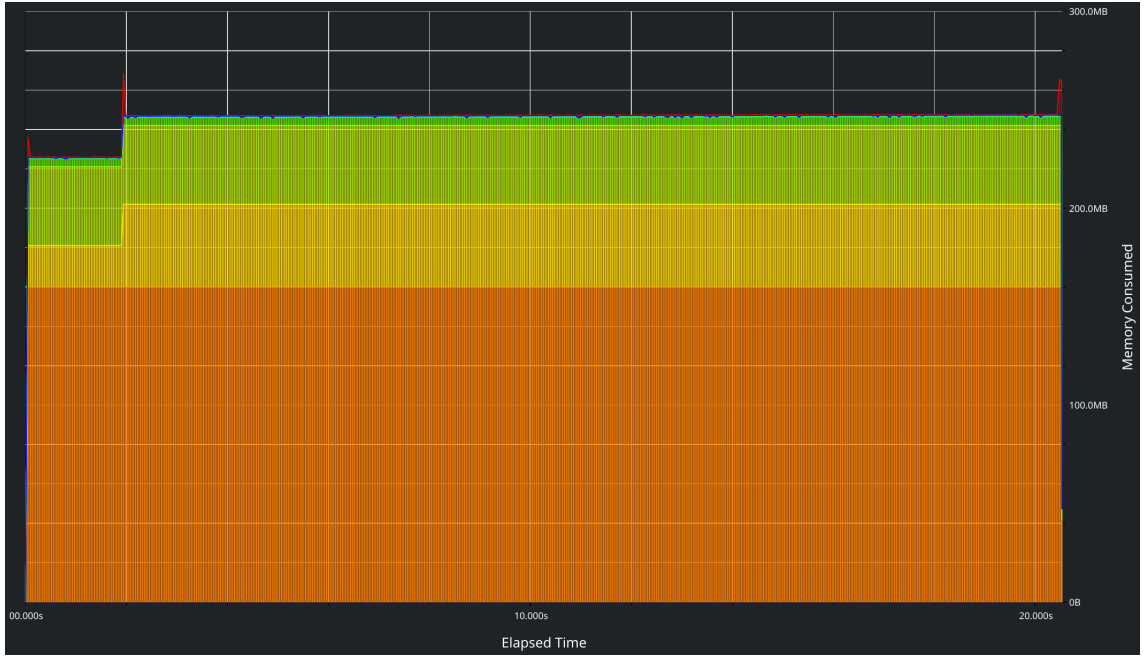


Figure 9.4: Heap consumption with the parquet writer enabled.

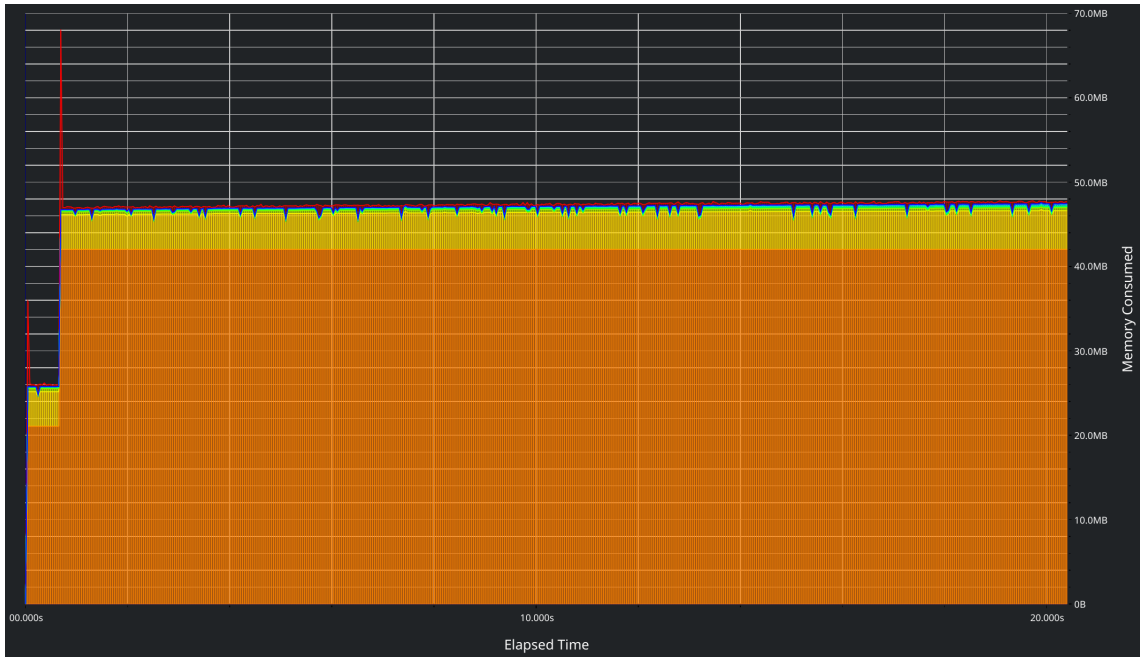


Figure 9.5: Heap consumption with the parquet writer removed. This run demonstrates that the entire core routine of the simulation, excluding the write buffer, can fit inside L3 cache.

Chapter 10

Conclusion

This thesis presented a high-throughput simulation architecture for photonic quantum network testbeds. The central idea is the separation of two concerns that existing simulators keep together: the tracking of which wavepackets are entangled with which, and the evaluation of the quantum states themselves. The tracking side is realized as a concurrent, lock-based architecture in which component nodes run on their own threads and record self-contained islands of interaction over the stream of wavepackets. Islands only ever merge, never split, and an island is collapsed exactly when its last packet terminates at a detector, which eliminates all cross-thread waiting from the pipeline.

On the representation side, this thesis developed a compact, transferable, universal operator format for the extracted islands. Photonic states are expressed over labeled modes, linear sections carry an error channel constrained to seven Kraus operators on the $\{|H\rangle, |V\rangle, |vac\rangle\}$ system, extending the qubit channel analysis of Ruskai et al. [6] with an absorbing vacuum level, and multi-port components carry occupied-mode reduced scattering matrices. The format is an interface contract: everything an engine needs to reconstruct and collapse the state of an island is contained in the record and the operator values it references. The receiving end of the contract beyond the in-process worker pool, such as an engine written in another language or running on another node of the cluster, has deliberately been left as future work. The sufficiency of the representation established in this thesis is what makes such implementations straightforward.

The architecture was evaluated end to end on a Hong–Ou–Mandel interference topology: one entangled photon pair source at 10^7 pairs per second, two optical delay lines, a beamsplitter, and two detectors with a realistic classical detector model (90% detection efficiency, 10^2 dark counts per second, and a 20 ns non-paralyzable dead time). A fine sweep of the arm length difference reproduces the interference dip with the spread expected from the wavepacket width, and the emitted time tags are written as a single, globally time-ordered Parquet stream in the same format as the laboratory time taggers. On a single workstation the simulator sustains a peak rate of 2.6×10^7 simulation events per second on six cores, and its single-core rate of 1.2×10^7 events per second already exceeds the single-threaded QuISP baseline of 1.1×10^7 packets per second measured on a zero-noise two-node link tomography workload, as shown in Chapter 9. The working set of the simulator is a few megabytes and stays constant over arbitrarily long runs, so the

whole simulation runs inside the processor cache.

10.1 Future Work

Several directions follow naturally from this work. First, the matrix engine side of the operator contract should be implemented, both as native engines in other languages and as remote engines on other nodes, turning the horizontal scalability argued in Chapter 9 into a measured result. Second, the placeholder identity channels should be replaced with Kraus operators and density matrices obtained from the tomography of the actual testbed components, so that the emitted time tags carry the error signatures of the real system. Third, the simulator output should be validated against the testbed measurements, following the methodology established for the module-based discrete-event simulator of Zhang et al. [3]; the shared time tag format makes the comparison direct, because the same analysis code can consume both streams. Finally, the scaling limit observed beyond six cores is set by contention on the shared interaction store, and relaxing it, for example by sharding the store across detector groups, is the clearest path to higher throughput on wider machines.

Bibliography

- [1] D. Sakuma et al., “Q-Fly: An Optical Interconnect for Modular Quantum Computers,” arXiv preprint arXiv:2412.09299, Dec. 2024, doi:10.48550/arXiv.2412.09299.
- [2] R. Satoh et al., “QuISP: A Quantum Internet Simulation Package,” in Proc. IEEE Int. Conf. on Quantum Computing and Engineering (QCE), 2022, pp. 353–364, doi:10.1109/QCE53715.2022.00056.
- [3] Y. Zhang, S. Nagayama, A. Soeda, S. Kawakami, T. Ono, and T. Tanimoto, “Experimental validation of module-based discrete-event simulator of quantum interconnects,” SIG Technical Reports, vol. 2025-ARC-263, no. 16, pp. 1–11, Dec. 2025.
- [4] K. Mato, M. Ringbauer, L. Burgholzer, and R. Wille, “MQT Qudits: A Software Framework for Mixed-Dimensional Quantum Computing,” arXiv preprint arXiv:2410.02854, 2024. doi: 10.48550/arXiv.2410.02854.
- [5] J. Maziero, “Computing partial traces and reduced density matrices,” International Journal of Modern Physics C, vol. 28, p. 1750005, 2017, doi: 10.1142/s012918311750005x
- [6] M. B. Ruskai, S. Szarek, and E. Werner, “An analysis of completely-positive trace-preserving maps on $\mathcal{M}_2$,” Linear Algebra and its Applications, vol. 347, pp. 159–187, 2002, doi:10.1016/S0024-3795(01)00547-X.
- [7] D. Manzano, “A short introduction to the Lindblad master equation,” AIP Advances, vol. 10, p. 025106, 2020, doi:10.1063/1.5115323.
- [8] M. A. Nielsen and I. L. Chuang, Quantum Computation and Quantum Information: 10th Anniversary Edition. Cambridge, UK: Cambridge University Press, 2010.
- [9] T. Coopmans et al., “NetSquid, a NETwork Simulator for QUantum Information using Discrete events,” Communications Physics, vol. 4, p. 164, 2021, doi:10.1038/s42005-021-00647-8.
- [10] Y. Zhang, S. Kawakami, T. Ono, and T. Tanimoto, “Design and Implementation of a Discrete-Event Simulator of Quantum Interconnect: a case for Bell-State Swapping,” IPSJ SIG Technical Report, vol. 2025-ARC-254, Aug. 2025.
- [11] P. Kok, W. J. Munro, K. Nemoto, T. C. Ralph, J. P. Dowling, and G. J. Milburn, “Linear optical quantum computing with photonic qubits,” Reviews of Modern Physics, vol. 79, pp. 135–174, 2007, doi:10.1103/RevModPhys.79.135.

- [12] S. D. Bartlett, T. Rudolph, and R. W. Spekkens, "Reference frames, superselection rules, and quantum information," *Reviews of Modern Physics*, vol. 79, pp. 555–609, 2007, doi:10.1103/RevModPhys.79.555.
- [13] V. V. Albert et al., "Performance and structure of single-mode bosonic codes," *Physical Review A*, vol. 97, p. 032346, 2018, doi:10.1103/PhysRevA.97.032346.
- [14] G. Cenedese, G. Benenti, and M. Bondani, "Correcting Coherent Errors by Random Operation on Actual Quantum Hardware," *Entropy*, vol. 25, no. 2, p. 324, 2023, doi:10.3390/e25020324.
- [15] B. Wu, X. Wang, X. Yuan, C. Huang, and J. Chen, "Leakage Benchmarking for Universal Gate Sets," *Entropy*, vol. 26, no. 1, p. 71, 2024, doi:10.3390/e26010071.
- [16] S. Koudia, L. Oleynik, M. Bayraktar, J. ur Rehman, and S. Chatzinotas, "Physical Layer Aspects of Quantum Communications: A Survey," *arXiv preprint arXiv:2407.09244*, 2024.
- [17] W. Vogels, "Eventually consistent," *Communications of the ACM*, vol. 52, no. 1, pp. 40–44, 2009, doi:10.1145/1435417.1435432.
- [18] U. Leonhardt, "Quantum physics of simple optical instruments," *Rep. Prog. Phys.*, vol. 66, pp. 1207–1249, 2003.
- [19] A. Aiello, G. Puentes, and J. P. Woerdman, "Linear optics and quantum maps," *Phys. Rev. A*, vol. 76, no. 3, p. 032323, Sep. 2007.
- [20] IBM, "Chi Class," in *Qiskit SDK API Reference*, 2026. [Online]. Available: https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Chi. [Accessed: Jul. 17, 2026].
- [21] M. Hajdušek and R. Van Meter, "Timing Regimes in Quantum Networks and their Physical Underpinnings," IETF, Internet-Draft draft-hajdusek-qirg-timing-physics-latest, Mar. 2026. [Online]. Available: <https://moonshot-nagayama-pj.github.io/draft-hajdusek-qirg-timing-physics/draft-hajdusek-qirg-timing-physics.html>. [Accessed: Jul. 17, 2026].
- [22] M. Reck, A. Zeilinger, H. J. Bernstein, and P. Bertani, "Experimental realization of any discrete unitary operator," *Physical Review Letters*, vol. 73, no. 1, pp. 58–61, 1994, doi:10.1103/PhysRevLett.73.58.
- [23] X. Wu et al., "SeQUeNCe: A customizable discrete-event simulator of quantum networks," *Quantum Science and Technology*, vol. 6, no. 4, p. 045027, 2021.
- [24] C. K. Hong, Z. Y. Ou, and L. Mandel, "Measurement of subpicosecond time intervals between two photons by interference," *Phys. Rev. Lett.*, vol. 59, no. 18, pp. 2044–2046, 1987.
- [25] R. L. Hudson, "When is the Wigner quasi-probability density non-negative?," *Reports on Mathematical Physics*, vol. 6, no. 2, pp. 249–252, 1974, doi:10.1016/0034-4877(74)90007-X.

- [26] S. Aaronson and A. Arkhipov, “The Computational Complexity of Linear Optics,” in *Proc. 43rd ACM Symposium on Theory of Computing (STOC)*, 2011, pp. 333–342, doi:10.1145/1993636.1993682.
- [27] H. Yokomori et al., “Evaluation of Distillation’s Real-world Performance on a Superconducting Quantum Computer,” arXiv preprint arXiv:2504.18141, 2025, doi:10.48550/arXiv.2504.18141.
- [28] F. Tonolini, S. Chan, M. Agnew, A. Lindsay, and J. Leach, “Reconstructing high-dimensional two-photon entangled states via compressive sensing,” *Scientific Reports*, vol. 4, p. 6542, 2014, doi:10.1038/srep06542.
- [29] P. Lenard, “Ueber die lichtelektrische Wirkung,” *Annalen der Physik*, vol. 313, no. 5, pp. 149–198, 1902, doi: 10.1002/andp.19023130510.
- [30] A. Einstein, “Ueber einen die Erzeugung und Verwandlung des Lichtes betreffenden heuristischen Gesichtspunkt,” *Annalen der Physik*, vol. 322, no. 6, pp. 132–148, 1905, doi: 10.1002/andp.19053220607.
- [31] M. Zirnbauer, *Quantenphysik*, Lecture Notes, Institut für Theoretische Physik, Universität zu Köln, Cologne, Germany, SS 2013. [Online]. Available: https://www.thp.uni-koeln.de/zirn/011_Website_Martin_Zirnbauer/3_Teaching/LectureNotes/All-QM_SS2013.pdf
- [32] P. W. Shor, “Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer,” *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, Oct. 1997, doi: 10.1137/S0097539795293172.
- [33] R. Van Meter, W. J. Munro, K. Nemoto, and K. M. Itoh, “Arithmetic on a Distributed-Memory Quantum Multicomputer,” *ACM Journal on Emerging Technologies in Computing Systems*, vol. 3, no. 4, Art. 13, 2008, doi: 10.1145/1322263.1322265.